

EFREI PARIS - M1 DATA ENGINEERING & IA

Maintenance Predictive Industrielle

150 Questions de Soutenance avec Reponses Detaillees

Bloc 2 - Competences C3.1 a C4.4

Adam Beloucif & Emilien Morice

Soutenance - Juin 2026

150 Questions - Reponses Detaillees - Preparation Jury

Section 1 - Formulation du probleme et dataset

C3.1 Objectif

1 Quel est l'objectif du projet de maintenance predictive ?

Predire si une machine industrielle tombera en panne dans les prochaines 24 heures (variable cible binaire `failure_within_24h`) a partir de capteurs en temps reel. L'objectif metier est de reduire les arrêts non planifies (downtime) en permettant une intervention preventive avant la panne. Le dataset comporte 24 042 enregistrements, 15 variables de capteurs, et environ 7% de pannes (desequilibre de classes). Quatre modeles ont ete compares, XGBoost etant retenu avec $F1=0.886$ et $ROC-AUC=0.995$.

C3.1 Horizon

2 Pourquoi `failure_within_24h` et pas `failure_within_1h` ou `48h` ?

Le choix de 24h est un compromis metier : 1) Horizon operationnel - les equipes maintenance ont besoin de 4-8h pour planifier et executer une intervention. 24h donne une marge suffisante. 2) Qualite predictive - sur ce dataset, les capteurs montrent des signaux precurseurs detectables 12-36h avant la panne. En dessous de 6h, le signal-bruit est trop eleve. Au-dela de 48h, la precision chute car trop d'inertie. 3) Alignement avec la pratique industrielle - les standards ISA-95 recommandent un horizon de 24-72h pour la maintenance predictive Niveau 2.

C3.1 Dataset

3 Decrivez le dataset : variables et distribution des classes.

15 variables d'entree : `vibration_rms` (g), `temperature_motor` (degC), `temperature_bearing` (degC), `current_draw` (A), `pressure_hydraulic` (bar), `oil_viscosity`, `rpm_speed`, `load_factor`, `maintenance_age_days` (jours depuis derniere maintenance), `cumulative_cycles`, `shock_count_24h`, `oil_temperature`, `voltage_supply`, `humidity_ambient`, `failure_within_24h` (cible). Distribution des classes : 93% negatif (pas de panne), 7% positif (panne imminente). Collecte via capteurs IIoT sur 6 mois de fonctionnement continu.

C3.1 Desequilibre

4 Qu'est-ce que le disequilibre de classes et comment l'avez-vous gere ?

Le disequilibre (7% positifs vs 93% negatifs) pose probleme car un modele naive qui predit toujours 'pas de panne' aurait 93% d'accuracy tout en etant inutile. Methodes appliquees : 1) SMOTE (Synthetic Minority Oversampling Technique) - genere des exemples positifs synthetiques par interpolation dans l'espace des features. 2) `class_weight='balanced'` dans XGBoost (`scale_pos_weight = 13.3 = 93/7`). 3) Metric de reference : F1-score macro et ROC-AUC plutot qu'accuracy. Le F1 macro penalise egalement la mauvaise prediction sur la classe minoritaire.

C3.1 Metriques

5 Pourquoi le F1-score et pas l'accuracy pour evaluer ?

L'accuracy sur un dataset disequilibre est trompeuse (93% sans rien predire). Le F1-score est la moyenne harmonique de la precision et du rappel. $Precision = TP/(TP+FP)$: parmi les pannes prevues, combien sont vraies ? $Rappel = TP/(TP+FN)$: parmi les vraies pannes, combien sont detectees ? En maintenance, un faux negatif (panne ratee) est plus couteux qu'un faux positif (intervention inutile). On pourrait donc privilegier le rappel en baissant le seuil de decision (default 0.5 -> 0.3). XGBoost avec threshold 0.35 atteint $Rappel=0.91$, $Precision=0.83$.

C3.2 Preprocessing

6

Quelles transformations de features avez-vous appliquees ?

Pipeline sklearn : 1) StandardScaler sur les variables numeriques continues (vibration_rms, temperatures, current) - centrage-reduction car XGBoost n'en a pas besoin mais les autres modeles (MLP, LogReg) si. 2) Winsorizing au percentile 1-99 pour les outliers de capteurs defectueux. 3) Feature engineering : ratio current_draw/rpm_speed (indice de charge effective), age_depuis_derniere_maintenance^2 (degradation non lineaire), rolling_mean_vibration_24h. 4) Pas d'imputation de NaN : les NaN de capteur sont codes -1 (valeur sentinelle) pour que XGBoost les traite comme une absence d'information.

C3.2 Split

7

Comment avez-vous separe les donnees sans leakage ?

Split temporel : on ne melange pas les observations aleatoirement (risque de leakage temporel - predire le passe a partir du futur). Le dataset est ordonne chronologiquement. 70% premiere periode -> train, 15% periode intermediaire -> validation, 15% periode finale -> test. Ce split respecte la causalite temporelle. De plus, les features rolling (ex: rolling_mean_24h) sont calculees uniquement sur l'historique precedant chaque observation - pas de lookahead bias. La cross-validation utilisee est TimeSeriesSplit (pas KFold classique).

C3.2 Leakage

8

Qu'est-ce que le leakage de donnees ?

Le leakage (ou data leakage) est quand des informations du futur ou de la cible se glissent dans les features d'entrenement, donnant des performances artificiellement elevees qui ne se generalisent pas en production. Exemples concrets : 1) Scaler fit sur tout le dataset (train + test) avant le split - le test 'connait' les stats du train. 2) Features calculees apres l'evenement (ex : 'maintenance effectuee dans les 24h' si inclusee comme feature). 3) Target leakage : une feature qui est causee par la panne plutot que cause de la panne. On l'evite en fitant le scaler uniquement sur le train et en appliquant le transform au test.

C3.2 Features

9

Quelles features ont ete le plus informatives ?

Selon l'analyse SHAP : 1) vibration_rms (contribution SHAP moyenne 0.43) - les vibrations anormales sont le signal precurseur le plus fiable. 2) maintenance_age_days (0.31) - plus la machine vieillit sans maintenance, plus le risque augmente non lineairement. 3) temperature_motor (0.28) - surchauffe systematiquement precedente des pannes. 4) shock_count_24h (0.19) - les chocs mecaniques cumulatifs fragilisent. 5) current_draw/rpm_speed ratio (0.17) - charge anormale. Ces 5 features expliquent 78% de la contribution SHAP totale.

C3.2 Selection

10

Avez-vous effectue une selection de features ?

Oui, via deux methodes : 1) Correlation de Pearson - temperature_bearing et temperature_motor correlees a 0.94 : on retire temperature_bearing (information redundante, risque de multicollinearite pour LogReg et MLP). 2) Feature importance XGBoost (gain moyen) : oil_viscosity et humidity_ambient ont importance < 0.01 et sont retires. Le modele final utilise 12 features sur 15. La suppression de features faibles ameliore legerement le F1 (+0.005) et reduit le temps d'inference de 15%.

C3.3 Benchmark

11

Quels modeles ont ete compares et pourquoi ce choix ?

4 modeles : 1) Regression Logistique (baseline lineaire, interpretable, F1=0.747). 2) Random Forest (ensemble arbres, robuste aux outliers, F1=0.863). 3) MLP/Reseau de neurones (2 couches cachees 128-64, ReLU, dropout 0.3, F1=0.836). 4) XGBoost (gradient boosting, F1=0.886, RETENU). Ce panel couvre la gamme classique ML supervisee : lineaire, ensemble bagging, reseau de neurones, ensemble boosting. On n'a pas teste SVM (lent sur 24K lignes) ni

LSTM (series temporelles, hors scope pour une prediction ponctuelle).

C3.3 RF vs XGB

12 Expliquez la difference entre Random Forest et XGBoost.

Random Forest : N arbres de decision entraines en PARALLELE sur des sous-echantillons bootstrap, vote majoritaire final (bagging). Chaque arbre est independant. XGBoost : N arbres entraines en SEQUENCE, chaque arbre corrige les erreurs du precedent (boosting). XGBoost minimise une fonction de perte differentiable via gradient descent. Resultats : RF est plus robuste au surapprentissage (bagging = variance reduction) mais moins precis. XGBoost est plus precis mais peut overfit si mal regularise (parametres max_depth, min_child_weight, lambda essentiels). XGBoost est generalement meilleur sur des donnees tabulaires.

C3.3 Hyperparams

13 Quels hyperparametres XGBoost avez-vous optimises ?

Recherche via Optuna (Bayesian optimization, 100 trials) sur 5-fold TimeSeriesSplit. Hyperparametres optimaux retenus : n_estimators=450, max_depth=6, learning_rate=0.08, subsample=0.85, colsample_bytree=0.75, min_child_weight=3, scale_pos_weight=13.3 (gestion desequilibre), reg_alpha=0.1 (L1), reg_lambda=1.5 (L2). Le learning_rate faible + n_estimators eleve ameliore la generalisation au prix d'un entraînement plus long (45s vs 8s pour les defaults). Metric optimisee : F1-macro sur le fold de validation.

C3.3 Choix Final

14 Pourquoi XGBoost est retenu avec F1=0.886 et pas Random Forest F1=0.863 ?

Trois raisons : 1) F1 superieur de 2.3 points - significatif pour la detection de pannes (chaque faux negatif = potentiellement un arret machine non planifie). 2) ROC-AUC = 0.995 vs 0.981 pour RF : XGBoost est bien meilleur pour discriminer toutes les classes de risque, utile si on veut afficher un score de risque continu. 3) Temps inference plus faible (8ms vs 23ms) : XGBoost utilise le format ONNX pour l'inference, plus rapide que Scikit-learn RF. La difference de 2.3 F1 justifie la complexite additionnelle et le cout CO2 legerement superieur.

C3.3 Overfitting

15 Comment avez-vous valide que le modele ne surapprenait pas ?

Deux verifications : 1) Comparaison train/validation/test : F1 train=0.918, validation=0.897, test=0.886 - ecart faible (< 3%), pas d'overfitting severe. 2) Learning curves : on trace F1 en fonction du nombre de donnees d'entraînement (10% a 100%). La courbe train descend (moins de donnees = plus easy a fitter) et la courbe validation monte, et les deux se rapprochent a 100% - signe d'un modele bien calibre. Regularisation L1/L2 appliquee apres avoir observe le gap train/val trop large avec les hyperparametres par default.

Section 2 - Exploration et preprocessing pipeline

C3.4 Gradient Boosting

16 Qu'est-ce que le gradient boosting mathématiquement ?

Le gradient boosting construit un modèle $F(x)$ en additionnant itérativement des arbres faibles. A l'itération t , on entraîne un arbre h_t sur les résidus du modèle précédent : $r_i = -dL/dF(x_i)$ (gradient négatif de la loss par rapport à la prédiction). Pour la loss binaire (cross-entropy), ce gradient est $y_i - p_i$ (résidus). Le nouveau modèle est $F_t(x) = F_{t-1}(x) + \eta * h_t(x)$ où η est le learning rate. XGBoost ajoute la régularisation L1/L2 sur les poids des feuilles dans la fonction de perte, ce qui le distingue du GBDT classique (Friedman 2001).

C3.4 ROC-AUC

17 Qu'est-ce que le ROC-AUC et pourquoi 0.995 est excellent ?

ROC (Receiver Operating Characteristic) : courbe qui trace le rappel (TPR) en fonction du taux de faux positifs (FPR) pour tous les seuils de décision. AUC (Area Under Curve) : aire sous cette courbe, entre 0 et 1. AUC=0.5 = modèle aléatoire, AUC=1.0 = perfection. AUC=0.995 signifie que pour 99.5% des paires (exemple positif, exemple négatif), le modèle attribue un score de risque plus élevé à l'exemple positif. En maintenance prédictive industrielle, un AUC ≥ 0.95 est considéré comme production-ready par les standards MIMOSA (Open Systems Architecture for Condition Based Maintenance).

C3.4 Matrice

18 Qu'est-ce que la matrice de confusion et comment interprétez-vous la votre ?

La matrice de confusion pour le test set (3606 observations, 7% positifs = ~252 pannes) : TP=229 (pannes détectées), TN=3306 (non-pannes correctes), FP=48 (fausses alarmes), FN=23 (pannes ratées). Interprétation : le modèle rate 23 pannes sur 252 (Rappel=0.91) et génère 48 fausses alarmes sur 3306 (FPR=1.5%). En contexte industriel, les 23 pannes ratées représentent le risque résiduel - chaque FN peut coûter 10-50K euros d'arrêt non planifié. Les 48 FP génèrent des interventions préventives inutiles mais leur coût est bien inférieur.

C3.4 Threshold

19 Comment fonctionne le threshold de décision et pourquoi 0.35 ?

XGBoost produit une probabilité continue entre 0 et 1 pour chaque observation. Le threshold (seuil) détermine à partir de quelle probabilité on classe en 'panne'. Default = 0.5 donne F1=0.886. En abaissant à 0.35 : Rappel monte de 0.91 à 0.94 (on manque moins de pannes), Precision descend de 0.87 à 0.81 (plus de fausses alarmes). Le seuil 0.35 est justifié métier : le coût asymétrique (FN $\gg$ FP) justifie de privilégier le rappel. Ce threshold est paramètre dans le fichier de configuration et ajustable selon le coût d'arrêt de la machine spécifique.

C3.4 FastAPI

20 Comment le modèle est-il servi en production via FastAPI ?

Le modèle XGBoost est serialisé en ONNX via `skl2onnx + xgboost.to_onnx()`. L'inférence utilise `onnxruntime` (10x plus rapide que le `predict()` sklearn natif). L'API FastAPI expose un endpoint POST `/predict` qui reçoit un JSON de 12 features, passe par le pipeline de preprocessing (StandardScaler chargé depuis joblib), puis appelle `onnxruntime` pour l'inférence. La réponse inclut : prédiction (0/1), probabilité, `risk_level` (LOW/MEDIUM/HIGH/CRITICAL selon seuils). Latence médiane : 8ms. Le modèle est versionné (MLflow) et le endpoint indique la version courante dans le header `X-Model-Version`.

C3.5 P/R

21 Qu'est-ce que la precision et qu'est-ce que le rappel ?

Precision = $TP / (TP + FP)$: sur toutes les pannes que le modele predit, quelle fraction est reellement une panne ? Haute precision = peu de fausses alarmes. Rappel (Sensitivity) = $TP / (TP + FN)$: sur toutes les vraies pannes, quelle fraction le modele detecte ? Haut rappel = peu de pannes ratees. Il existe un trade-off : augmenter le rappel (seuil plus bas) augmente les faux positifs (baisse precision). $F1 = 2 * Precision * Rappel / (Precision + Rappel)$ est la moyenne harmonique qui equilibre les deux. On preferera maximiser le rappel quand le cout d'un FN est eleve (ici oui).

C3.5 CV Temporelle

22 Qu'est-ce que la validation croisee temporelle (TimeSeriesSplit) ?

La cross-validation classique (K-Fold) melange aleatoirement les observations, ce qui cree du leakage temporel (utiliser le futur pour predire le passe). TimeSeriesSplit respecte l'ordre chronologique : Fold 1 : train $[t0, t1]$, val $[t1, t2]$. Fold 2 : train $[t0, t2]$, val $[t2, t3]$. Fold 3 : train $[t0, t3]$, val $[t3, t4]$. Chaque fold agrandit le train et avance dans le temps. Le score final est la moyenne des F1 sur les 5 folds de validation - donne une estimation fiable de la generalisation temporelle.

C3.5 Comparaison

23 Comment avez-vous compare les 4 modeles de maniere rigoureuse ?

Protocole standardise : meme split temporel (70/15/15%), meme preprocessing pipeline, meme metric (F1-macro, ROC-AUC), meme seed random. Chaque modele a ete optimise independamment via Optuna (100 trials, TimeSeriesSplit 5 folds). Les scores reportes sont sur le test set final, jamais touche pendant l'optimisation des hyperparametres. Tableau recapitulatif : LogReg F1=0.747/AUC=0.912, RF F1=0.863/AUC=0.981, MLP F1=0.836/AUC=0.967, XGBoost F1=0.886/AUC=0.995. Le test de McNemar confirme que la difference XGBoost/RF est statistiquement significative ($p < 0.01$).

C4.1 SHAP

24 Qu'est-ce que SHAP et pourquoi est-ce important pour la maintenance ?

SHAP (SHapley Additive exPlanations) est une methode d'explicabilite basee sur la theorie des jeux (valeurs de Shapley). Pour chaque prediction, SHAP calcule la contribution de chaque feature a l'ecart entre la prediction et la prediction moyenne. Importance pour la maintenance : 1) Confiance operatrice - les techniciens veulent comprendre POURQUOI le modele sonne l'alarme avant d'intervenir. 2) Debugging - si vibration_rms a une contribution negative anormale, c'est peut-etre un capteur defaillant. 3) Conformite industrielle - certaines normes (IEC 61511) exigent la tracabilite des decisions d'arret.

C4.1 SHAP Global/Local

25 Quelle est la difference entre SHAP global et local ?

SHAP global : agregation des valeurs SHAP absolues sur tout le dataset. Montre l'importance relative des features en moyenne. Ex : vibration_rms est la feature la plus importante sur les 24K observations. SHAP local : valeurs SHAP pour une prediction specifique. Ex : pour la machine ID-5723 a 14h30, la prediction 'PANNE' est due a +0.43 vibration_rms, +0.31 maintenance_age_days, -0.08 temperature_motor. L'operateur voit exactement quels capteurs ont declenche l'alarme. On affiche la SHAP locale dans l'interface Streamlit pour chaque alerte.

C4.1 SHAP Insights

26 Quelles conclusions tirez-vous des valeurs SHAP ?

Trois insights actionnables : 1) vibration_rms domine les pannes mecaniques (rodements, balancement) - renforcer la surveillance vibratoire en priorite. 2) maintenance_age_days a une contribution non-lineaire : faible jusqu'a 180 jours, puis croissante exponentiellement - recommande un intervalle de maintenance preventive de 150 jours maximum. 3) temperature_motor et current_draw sont correlees dans les SHAP : une anomalie thermique sans anomalie electrique pointe vers un probleme de refroidissement vs une surcharge. Ces insights ont ete valides par les experts maintenance

de l'entreprise partenaire.

C4.1 SHAP Performance

27 Est-ce que SHAP ralentit le systeme en production ?

Oui, le calcul SHAP complet (background dataset 1000 obs, TreeExplainer) prend ~200ms par prediction. C'est acceptable pour une consultation post-alarme mais pas pour un monitoring en continu. En production, on utilise FastTreeSHAP (version optimisee, ~15ms) et on calcule les SHAP uniquement quand la probabilite de panne > 0.35 (threshold d'alarme). Les predictions < 0.35 retournent uniquement le score, pas les SHAP. Cela reduit de 90% le volume de calculs SHAP (seulement ~7% des cas declenchent le calcul).

C4.2 CodeCarbon

28 Qu'est-ce que CodeCarbon et pourquoi l'avez-vous utilise ?

CodeCarbon est une bibliotheque Python qui mesure la consommation electrique du code CPU/GPU et la convertit en grammes de CO2 equivalent en fonction du mix energetique local (localisation IP -> region -> facteur carbone kgCO2/kWh). On l'a integre pour mesurer l'empreinte carbone de l'entrainement des 4 modeles. Resultat : XGBoost empreinte entrainement = 6.1mg CO2eq (Optuna 100 trials inclus), RF = 8.2mg CO2eq (les 450 arbres independants sont plus couteux), MLP = 4.3mg CO2eq, LogReg = 0.8mg CO2eq. Ces mesures s'inscrivent dans la demarche de Green AI / IA responsable.

C4.2 CO2 Comparaison

29 Pourquoi XGBoost a une empreinte moindre que RF malgre de meilleures performances ?

XGBoost construit 450 arbres en sequence, chaque arbre etant petit (max_depth=6, pruning agressif). Random Forest construit 500 arbres en parallele (tous les CPU occupez), chacun plus profond par design (no depth limit par default). RF consomme plus car il exploite la parallelisation CPU intensive sur plus de temps. XGBoost est plus efficace car chaque arbre est construit sur les residus - les arbres tardifs sont tres petits (ils corrigent de petites erreurs). De plus, XGBoost utilise un histogram-based split finding qui est bien plus rapide que l'exact split de RF.

C4.2 Implementation

30 Comment integrez-vous CodeCarbon dans votre code ?

Avec le context manager : `from codecarbon import EmissionsTracker; with EmissionsTracker(output_dir='reports/') as tracker: model.fit(X_train, y_train)`. A la sortie du context, `tracker.final_emissions` contient les g CO2eq. Les resultats sont sauvegardes dans `emissions.csv` avec : `timestamp`, `projet`, `modele`, `energie_kwh`, `emissions_gco2eq`, `pays`, `region`. Ce fichier est commite dans le repo pour tracabilite. Pour l'inference en production, on ne mesure pas en continu (overhead 2ms/call) mais on fait une mesure periodique (1/1000 predictions) pour surveiller la derive.

Section 3 - Benchmark 4 modeles - comparaison rigoureuse

C4.3 Streamlit

31 Decrivez l'interface Streamlit du dashboard maintenance.

Interface en 3 onglets : 1) Dashboard temps reel - tableau des N dernieres predictions avec code couleur (vert/orange/rouge/rouge vif selon risk_level), graphes de serie temporelle pour les 5 features SHAP top. 2) Machine specifique - historique 30 jours pour une machine selectionnee, courbe de probabilite de panne, SHAP waterfall de la derniere prediction. 3) Performance modele - matrice de confusion, ROC curve, F1 historique sur 30 derniers jours en production (monitoring derive). L'interface se rafraichit toutes les 30 secondes via st.rerun().

C4.3 Drift

32 Comment surveillez-vous la derive du modele en production ?

Monitoring via deux mecanismes : 1) Data drift - KS test (Kolmogorov-Smirnov) quotidien sur la distribution de chaque feature vs la distribution d'entrainement. Si $KS > 0.2$ sur 2+ features, alerte mail. 2) Concept drift - F1 hebdomadaire sur les labels vrais recus (avec delai de 24h pour confirmer si la panne a eu lieu). Si $F1 < 0.80$ sur 2 semaines consecutives, declenchement d'un retraining automatique sur les 3 derniers mois. Ces mecanismes evitent la degradation silencieuse des performances apres un changement de comportement machine.

C4.3 Streamlit Justification

33 Pourquoi Streamlit et pas une application React full-stack ?

Streamlit est justifie pour un contexte academique et prototype : (1) Developpement 5x plus rapide - une application de monitoring en ~100 lignes Python vs ~500 lignes React+API. (2) Pas besoin de build pipeline - modification = rechargement. (3) Natif avec l'ecosysteme Python (pandas, plotly, shap). Limites : Streamlit ne scale pas bien avec plusieurs utilisateurs simultanees (session state non partagee, re-run complet a chaque interaction). En production industrielle, on migrerait vers une vraie app React/Next.js avec API FastAPI pour le backend, comme sur Urban Data Explorer.

C4.4 Biais

34 Quels biais potentiels existe-t-il dans ce projet ?

Trois biais identifies : 1) Biais de survie - le dataset ne contient que les machines qui ont continue a fonctionner assez longtemps pour etre dans le dataset. Les machines tombees en panne catastrophique du premier jour sont absentes. 2) Biais de confirmation - les capteurs ont peut-etre ete installes APRES que les techniciens ont commence a observer les precurseurs, induisant un biais de selection temporel. 3) Biais de distribution - 6 mois de donnees en conditions normales, pas de donnees de stress test (ex : ete caniculaire, charge maximale prolongee). Le modele peut underperformer dans ces conditions extremes.

C4.4 Generalisation

35 Peut-on generaliser ce modele a d'autres types de machines ?

Non sans retraining. Le modele est specifique au type de machine du dataset (compresseurs industriels, d'apres la description des variables). Les signaux precurseurs (vibration, temperature, pression hydraulique) et leurs seuils de criticite sont specifiques a la mecanique de cette machine. Generaliser sans retraining sur une fraiseuse CNC par exemple donnerait de mauvaises predictions. La bonne pratique est de traiter ce modele comme un template : meme pipeline, meme hyperparametres de depart, mais retraining sur les donnees de la nouvelle machine avec au moins 3-6 mois d'historique.

C4.4 Capteur Panne

36 Que se passe-t-il si un capteur tombe en panne ?

Deux scenarios : 1) Capteur NaN - les features manquantes sont codees -1 (valeur sentinelle hors plage physique). XGBoost gere nativement les valeurs manquantes via ses regles de branchement appris. Les tests unitaires couvrent ce cas (toutes les predictions restent valides avec 1-3 features manquantes, F1 degrade de ~0.03). 2) Capteur bloque (valeur constante) - plus difficile a detecter. Un check de variance est fait toutes les heures : si $\text{ecart-type}(24h) < 0.001 * \text{ecart-type historique}$, alerte 'capteur suspect'.

C4.4 Ameliorations

37 Comment amelioreriez-vous le modele ?

Trois pistes : 1) Modelisation de la degradation continue - plutot qu'un label binaire, predire le Remaining Useful Life (RUL) en jours - regression plutot que classification. XGBoost peut etre adapte, mais un LSTM sur la serie temporelle des capteurs serait plus naturel. 2) Anomaly detection non supervisee (Isolation Forest, VAE) en parallele pour detecter les modes de panne jamais vus en entraînement. 3) Calibration des probabilites (Platt scaling ou isotonic regression) pour que la probabilite 0.7 corresponde vraiment a 70% de risque - utile pour la communication des risques aux operateurs.

C4.4 RNCP C3

38 Quelles sont les competences RNCP Bloc 2 C3 demontrees ?

C3.1 : formulation du probleme ML supervisee, choix du dataset (24K obs, 15 variables, 7% desequilibre), definition des metriques (F1, ROC-AUC). C3.2 : preprocessing pipeline (StandardScaler, SMOTE, feature engineering, split temporel sans leakage). C3.3 : benchmark de 4 modeles (LogReg, RF, MLP, XGBoost), hyperparameter tuning via Optuna. C3.4 : analyse SHAP (global + local), explicabilite des decisions. C3.5 : evaluation rigoureuse (test McNemar, learning curves, monitoring derive). Demonstration complete du cycle de vie ML de la collecte au deploiement en production.

C4.4 Valeur Industrielle

39 En quoi ce projet a-t-il une valeur industrielle reelle ?

Quatre elements de valeur : 1) ROI mesurable - avec Rappel=0.91 et 252 pannes detectees sur 276 en test, on evite 229 arrêts non planifies. Si chaque arret coute 20K euros (moyenne industrie discrete), le ROI brut est 4.58M euros/an. 2) Explicabilite SHAP - contrairement a un black box, les techniciens comprennent et font confiance au modele. 3) Empreinte carbone tracee (CodeCarbon) - repondre aux criteres ESG des grands groupes industriels. 4) Architecture deployable immediatement - FastAPI + ONNX + Streamlit est un stack production-ready, pas juste un notebook Jupyter.

C4.4 Equite

40 Comment gerer l'equite et les biais algorithmiques dans ce contexte ?

Dans la maintenance predictive, l'equite ne concerne pas les groupes humains mais les types de machines ou les sites de production. Un biais potentiel : si le dataset surrepresente les machines d'un site specifique, le modele sera moins performant sur les autres sites (biais de representativite). Mesures : (1) analyse stratifiee des performances par site et par type de machine, (2) audit de fairness trimestriel : F1 par sous-groupe doit rester dans 5% de la performance globale, (3) retraining si un sous-groupe specifique degrade. Pour les futurs projets impliquant des decisions sur des personnes, les memes principes s'appliquent avec en plus les obligations RGPD.

C4.4 Deep Learning

41 Quelle aurait ete la valeur ajoutee d'un modele de deep learning ?

Un LSTM ou Transformer sur la serie temporelle brute des capteurs (fenetres glissantes de 24h, granularite 1min = 1440 timesteps) pourrait capturer des patterns temporels complexes que XGBoost rate : degradations progressives sur plusieurs jours, patterns cycliques (usure plus rapide en debut de semaine apres arret weekend). Cependant : 1) Le dataset de 24K observations est trop petit pour entrainer un DL efficacement (besoin de 100K+ pour les LSTM). 2) La

latence d'inference LSTM est 5-10x supérieure. 3) L'explicabilité est bien moins bonne que SHAP sur XGBoost. XGBoost est le bon compromis pour cette taille de dataset.

C3.2 SMOTE

42 Qu'est-ce que SMOTE et comment fonctionne-t-il ?

SMOTE (Synthetic Minority Oversampling Technique) génère des exemples synthétiques de la classe minoritaire. Algorithme : pour chaque exemple positif x_i , on trouve ses K plus proches voisins dans la classe positive, on en choisit un x_{nn} aléatoirement, et on crée un nouvel exemple : $x_{new} = x_i + \lambda * (x_{nn} - x_i)$ où λ est uniforme $[0,1]$. Cela interpole entre des exemples réels plutôt que de simplement dupliquer (oversampling naïf). Après SMOTE, le dataset d'entraînement est équilibré (50/50). Important : SMOTE est appliqué UNIQUEMENT sur le train set, jamais sur le validation/test (évite le leakage d'augmentation).

C3.5 ROC Courbe

43 Qu'est-ce que la courbe ROC et comment la tracer ?

La courbe ROC (Receiver Operating Characteristic) trace le Rappel ($TPR = TP/(TP+FN)$) en fonction du FPR ($FP/(FP+TN)$) pour tous les seuils de décision possible (de 0 à 1). Pour chaque seuil, on calcule (FPR, TPR) et on trace un point. La courbe part de (0,0) (seuil=1, tout classe négatif) à (1,1) (seuil=0, tout classe positif). L'aire sous la courbe (AUC) mesure la capacité globale de discrimination. En Python : `from sklearn.metrics import roc_curve, auc; fpr, tpr, thresh = roc_curve(y_test, y_proba); auc_score = auc(fpr, tpr)`. On trace avec matplotlib et on affiche sur le dashboard Streamlit.

C3.4 Arbre XGB

44 Qu'est-ce qu'un arbre de décision et comment XGBoost en construit-il 450 ?

Un arbre de décision partitionne récursivement l'espace des features en régions homogènes. Chaque nœud interne : 'si vibration_rms > 2.3 alors gauche sinon droite'. Chaque feuille donne une valeur de prédiction. XGBoost construit 450 arbres en séquence, chaque arbre visant à corriger les erreurs du précédent. Le premier arbre prédit les résidus bruts, le deuxième prédit les résidus du premier, etc. La prédiction finale est la somme pondérée ($learning_rate=0.08$) de tous les arbres. Chaque arbre est contraint ($max_depth=6$) pour éviter l'overfitting. XGBoost utilise des histogrammes discrets des valeurs de features pour trouver les splits rapidement.

C3.4 Regularisation

45 Qu'est-ce que la regularisation L1 et L2 dans XGBoost ?

L1 (Lasso, $reg_alpha=0.1$) : pénalité proportionnelle à la valeur absolue des poids des feuilles. Encourage la sparsification - force certains poids à exactement 0 (sélection de features implicite). L2 (Ridge, $reg_lambda=1.5$) : pénalité proportionnelle au carré des poids. Lisse les poids sans les annuler. Dans XGBoost, ces regularisations sont intégrées directement dans la fonction objectif d'optimisation (pas ajoutées post-entraînement). Effet pratique : sans regularisation, le modèle overfit le train set. Avec $reg_lambda=1.5$, le gap train/val passe de 7% à 3%.

Section 4 - XGBoost - hyperparametres et resultats retenus

C3.3 Optuna

46 Qu'est-ce qu'Optuna et comment l'avez-vous utilise pour le tuning ?

Optuna est un framework d'hyperparameter optimization utilisant l'optimisation bayesienne (TPE - Tree-structured Parzen Estimator). Contrairement a GridSearch (exploration exhaustive) ou RandomSearch (exploration aleatoire), Optuna construit un modele probabiliste de la relation hyperparametres->performance et propose des combinaisons intelligentes (exploite les zones prometteuses). Code : `study = optuna.create_study(direction='maximize');`
`study.optimize(objective, n_trials=100)`. La fonction objective entraine XGBoost sur les hyperparametres proposes et retourne le F1 de validation. 100 trials Optuna >> 100 trials RandomSearch pour la meme qualite finale.

C3.4 Subsampling

47 Qu'est-ce que le sous-echantillonnage (subsampling) dans XGBoost ?

`subsample=0.85` signifie que chaque arbre est entraine sur un echantillon aleatoire de 85% des observations (sans remplacement). Cela reduit la variance (les arbres voient des sous-ensembles differents) et accelere l'entrainement. `colsample_bytree=0.75` fait la meme chose sur les features : chaque arbre voit 75% des features aleatoirement. Ces deux mecanismes sont similaires au Random Forest mais dans le contexte du boosting. Ils evitent que tous les arbres convergent vers le meme pattern (diversite des arbres = meilleure generalisation). Avec `subsample=1.0` et `colsample=1.0`, XGBoost est plus lent et overfit davantage.

C3.3 KFold Interdit

48 Qu'est-ce que la cross-validation k-fold classique et pourquoi ne pas l'utiliser ici ?

K-Fold CV divise le dataset en K parties egales. Pour chaque fold k : train sur les K-1 autres parties, validation sur la partie k. Score = moyenne des K scores. Probleme pour les series temporelles : si on melange aleatoirement les observations, une observation de mai peut etre dans le train et une de janvier dans le test. Le modele 'voit le futur' pendant l'entrainement (leakage temporel). Avec `TimeSeriesSplit`, les folds respectent l'ordre chronologique : on ne peut jamais utiliser une observation future pour predire le passe. C'est critique pour la maintenance predictive ou la saisonnalite et les derives temporelles sont reelles.

C4.3 Drift

49 Comment detectez-vous que le modele derive en production ?

Deux approches complementaires : 1) Monitoring des features (data drift) - KS test (Kolmogorov-Smirnov) hebdomadaire : compare la distribution de chaque feature en production vs distribution d'entrainement. Si la statistique $KS > 0.2$ pour 3+ features, alerte. 2) Monitoring du label (concept drift) - chaque semaine, on compare le F1 recalcule sur les labels vrais collectes avec 24h de delai. Si $F1 < 0.80$ (seuil defini), on declenche un retraining automatique sur les 3 derniers mois. Ces mecanismes sont implements dans un script `monitoring.py` execute par cron quotidiennement.

C4.3 ONNX

50 Qu'est-ce que ONNX et pourquoi l'utilisez-vous pour l'inference ?

ONNX (Open Neural Network Exchange) est un format intermediaire pour les modeles ML. `xgboost.to_onnx()` exporte le modele entraine en un graphe ONNX. `onnxruntime` l'execute via une runtime C++ ultra-optimisee. Avantages vs `predict()` sklearn/xgboost natif : 1) 3-5x plus rapide (operations SIMD, parallelisme CPU). 2) Portable - le meme fichier .onnx tourne sur Windows/Linux/Mac, en Python, C++, Java, C#. 3) Versionne facilement - le fichier .onnx est un artefact MLflow. 4) Pas de dependance xgboost sur le serveur de production - juste onnxruntime (plus leger). Latence mesure : `predict()` xgboost = 25ms, `onnxruntime` = 8ms pour 1000 predictions.

C4.3 MLflow

51 Qu'est-ce que MLflow et l'avez-vous utilise ?

MLflow est une plateforme open-source pour le lifecycle ML : tracking (log des hyperparametres, metriques, artefacts a chaque run), model registry (versionning des modeles), serving. Dans notre projet : `mlflow.xgboost.autolog()` logge automatiquement les hyperparametres et metriques a chaque run. Le meilleur modele ($F1=0.886$) est enregistre dans le model registry sous le nom 'predictive_maintenance_v1'. L'API FastAPI charge le modele depuis le registry via `mlflow.pyfunc.load_model('models:/predictive_maintenance/Production')`. Cela permet de rollback vers une version precedente en cas de probleme sans redeploy.

C4.3 Retrainement

52 Comment gerez-vous le reentrainement du modele ?

Pipeline de reentrainement declenchable manuellement ou automatiquement (si derive detectee) : 1) Extraction des 3 derniers mois de donnees avec labels vrais confirmes. 2) Preprocessing identique au pipeline initial (StandardScaler fit sur les nouvelles donnees, SMOTE). 3) Optuna fine-tuning (20 trials, pas 100 - on repart des meilleurs hyperparametres precedents). 4) Comparaison : si $F1_{nouveau} \geq F1_{actuel} - 0.01$ (seuil de stabilite), deploiement du nouveau modele. 5) Rollback automatique si les metriques de monitoring degradent dans les 48h suivant le deploiement.

C3.5 Calibration

53 Qu'est-ce que la calibration de probabilite et est-elle necessaire ?

La calibration verifie que les probabilites predites sont bien calibrees : un modele 'calibre' qui predit 70% de probabilite doit avoir raison 70% du temps. XGBoost n'est pas nativement bien calibre (il optimise le rang des predictions, pas la calibration). Verification : diagramme de fiabilite (reliability diagram) - on groupe les predictions par intervalles de probabilite et on trace la probabilite reelle vs predite. Si la courbe est loin de la diagonale, on applique la calibration de Platt (regression logistique sur les scores) ou isotonic regression. Dans notre cas, le diagramme montre une bonne calibration entre 0.3 et 0.7 (zone de decision), suffisant.

C4.1 SHAP Valeur

54 Comment interpretez-vous une valeur SHAP individuelle ?

SHAP value de 0.43 pour vibration_rms sur une prediction specifique signifie : 'la valeur de vibration_rms observee (ex: 3.2g) a augmente la log-odds de panne de 0.43 par rapport a la prediction moyenne du modele'. Les valeurs positives poussent vers la panne (classe 1), negatives vers la non-panne. La somme de tous les SHAP values + la valeur de base (prediction moyenne globale) = log-odds de la prediction finale. En pratique, l'interface Streamlit affiche un waterfall chart SHAP : la barre rouge montre les features qui poussent vers 'PANNE', verte vers 'OK', classees par magnitude.

C4.1 Gini vs SHAP

55 Quelle est la difference entre importance SHAP et importance Gini de XGBoost ?

Importance Gini (gain) de XGBoost : calculee pendant l'entrainement, mesure la reduction moyenne d'impurete (entropie) apportee par chaque feature sur tous les splits. Avantage : rapide (pas de calcul supplementaire). Inconvenient : peut etre biaisee vers les features avec plus de valeurs uniques ou de haute cardinalite. Importance SHAP : calculee post-entrainement, mesure la contribution marginale de chaque feature pour chaque prediction (theoreme Shapley). Plus precis, non biaise par la cardinalite, fournit une explication locale (par prediction) et globale (agregation). Recommandation : toujours preferer SHAP pour l'explication finale, Gini pour un diagnostic rapide pendant le dev.

C4.4 ROI

56 Comment avez-vous mesure l'impact economique du projet ?

Estimation ROI : (1) Cout d'un arret non planifie = 20K euros (donnee industrielle standard, secteur mecanique discrete). (2) Sur 3606 observations test : 229 vrais positifs (pannes correctement predites). Si on assume 1 observation test = 1 jour de production = 1 potentielle panne, ROI brut = $229 * 20K = 4.58M$ euros/an. (3) Cout intervention preventive faux positif = 500 euros (2h technicien). $48 FP * 500 = 24K$ euros. (4) ROI net = $4.58M - 24K = 4.556M$ euros/an. Ces chiffres sont des estimations pour valoriser le projet - une etude economique rigoureuse necessiterait des donnees operationnelles reelles.

C4.4 Ethique

57 Quelles sont les implications ethiques d'un systeme predictif industriel ?

Trois dimensions ethiques : 1) Confiance operatrice - si les techniciens ne comprennent pas pourquoi le modele alerte (black box), ils peuvent ignorer les alertes. SHAP et l'explicabilite sont cruciaux pour l'adoption. 2) Responsabilite - si le modele manque une panne (FN), qui est responsable ? Il faut maintenir un audit trail des predictions et des decisions humaines. Le modele est un outil d'aide a la decision, pas un decisionnaire autonome. 3) Impact emploi - la maintenance predictive peut reduire le nombre de techniciens necessaires. Discussion de cette implication avec les parties prenantes (syndicats, management) necessaire avant deploiement. Notre rapport inclut une section 'Impacts sociaux'.

C4.4 Model Card

58 Comment gerez-vous la documentation du modele ML ?

Artefacts de documentation ML : 1) Model Card (pratique Google) - fiche de 2 pages decrivant le modele : objectif, dataset, metriques, biais connus, limitations, recommandations d'usage. 2) README MLflow - description des hyperparametres, procedure de retraining. 3) Notebook EDA (Exploratory Data Analysis) - analyse du dataset, distributions, correlations, visualisations. Commite dans docs/notebooks/. 4) ARCHITECTURE.md - diagramme du pipeline complet de la collecte au deploiement. 5) runbooks/maintenance.md - procedures operationnelles (retraining, rollback, monitoring).

C3.2 Pipeline

59 Qu'est-ce qu'un pipeline scikit-learn et pourquoi l'utiliser ?

`sklearn.Pipeline` enchaîne des transformateurs et un estimateur final dans un objet unique : `pipe = Pipeline([('scaler', StandardScaler()), ('smote', SMOTE()), ('model', XGBClassifier())])`. Avantages : 1) Evite le leakage - le `fit()` du pipeline sur le train set applique automatiquement le fit de chaque etape avant de passer au suivant. Le scaler est fit sur le train uniquement. 2) Serialisation - le pipeline entier (scaler + modele) est sauvegarde en un seul fichier `joblib`. 3) `GridSearch/Optuna` peut optimiser les hyperparametres de n'importe quelle etape du pipeline. Dans notre projet, le Pipeline integre `StandardScaler` + `XGBoost` (`SMOTE` est applique separement car incompatible avec les pipelines `sklearn standard`).

C3.2 Features Temp

60 Comment avez-vous traite les features temporelles ?

Variables derivees du temps : 1) `maintenance_age_days` : calculee comme `(current_date - last_maintenance_date)`. Simple mais tres informative (SHAP rank 2). 2) `cumulative_cycles` : compteur cumulatif depuis la mise en service - mesure l'usure totale. 3) Features rolling window : `rolling_mean_vibration_24h`, `rolling_std_temperature_6h` - capturent la tendance recente (non juste la valeur instantanee). 4) Heure de la journee (`hour_of_day`) et jour de la semaine (`day_of_week`) - les machines ont des patterns de charge differents selon les shifts. Ces features ameliorent le F1 de 0.863 (sans) a 0.886 (avec).

Section 5 - Metriques F1, ROC-AUC, matrice de confusion

C4.3 Lifecycle

61 Comment le modele est-il maintenu a jour dans le temps ?

Cycle de vie ML (MLOps) : Mensuel - retraining sur rolling window 6 mois glissants (vieux data drops). Trimestriel - audit complet : test de McNemar pour comparer nouveau vs ancien modele, revue des SHAP pour detecter des changements de patterns (ex: une nouvelle piece mecanique rend une feature obsolete). Annuel - evaluation de l'architecture (envisager LSTM si le dataset atteint 200K+ obs). Les modeles sont versionnes v1.0, v1.1, etc. dans MLflow. Le changement de version majeure (architecture differente) necessite une validation humaine avant deploiement.

C4.3 Infrastructure

62 Quelles ressources computationnelles sont necessaires en production ?

Ressources minimales pour une instance de production : CPU 4 vcores (inference ONNX parallelisee), RAM 4GB (modele XGBoost ~50MB + onnxruntime + FastAPI), Stockage 10GB (MLflow artifacts, logs). Inference : 1 prediction en 8ms -> capacite theorique 125 predictions/seconde par instance. Pour 1000 machines surveillees avec update toutes les 10 secondes = 100 req/s -> 1 instance CPU largement suffisante. Pour 10 000 machines -> 2 instances en load balancing. Pas besoin de GPU (inference XGBoost est CPU-only et rapide). CodeCarbon inference : ~0.01mg CO2 par prediction.

C4.4 Prod Ready

63 Quelle est votre definition de 'pret pour la production' pour ce modele ?

Checklist production-ready : F1 ≥ 0.85 sur le test set (atteint : 0.886), Rappel ≥ 0.88 (atteint : 0.91), Latence inference p99 < 50ms (atteint : 8ms), Tests unitaires et integration couvrant > 80% du code (atteint), Documentation model card complete (atteint), Monitoring derive configure (atteint - KS test + F1 rolling), Procedure de retraining documentee et testee (atteint), Approbation des experts metier sur les resultats SHAP (atteint - session de validation avec techniciens). Manquant : tests de charge (simulation 1000 requetes/s), audit de securite de l'API, approbation RSSI.

C3.3 Reproductibilite

64 Comment avez-vous gere la reproductibilite des experimentations ?

Trois niveaux de reproductibilite : 1) Seed random fixe - random_state=42 partout (sklearn, numpy, XGBoost, SMOTE). Permet de reproduire exactement les memes resultats si on relance le meme code. 2) Environnement fixe - requirements.txt avec versions exactes (xgboost==2.1.0, scikit-learn==1.5.1, etc.), mais surtout requirements.lock generee via pip-compile. 3) MLflow tracking - chaque run est loggue avec les hyperparametres, les metriques, et le code exact (git commit hash). On peut re-executer exactement n'importe quel run depuis MLflow. Sans ces pratiques, deux chercheurs peuvent obtenir des resultats differents sur le meme code.

C3.5 Learning Curves

65 Qu'est-ce que la courbe d'apprentissage et qu'en avez-vous conclu ?

La courbe d'apprentissage trace les scores train et validation en fonction de la taille du dataset d'entrainement (de 10% a 100%). Lecture : 1) Si train $\gg$ validation : overfitting (plus de donnees ou plus de regularisation necessaire). 2) Si train $\sim$ validation mais les deux sont bas : underfitting (modele trop simple ou pas assez de features). 3) Si les courbes convergent et se rejoignent en haut : bonne generalisation. Notre courbe : a 10% train (1700 obs), F1 train=0.95, F1 val=0.82 (gros gap = overfitting). A 100% (17K obs), F1 train=0.918, F1 val=0.897 (gap reduit = bonne generalisation). Conclusion : plus de donnees continueraient d'ameliorer le modele.

C3.1 Dataset Justif

66

Comment avez-vous justifie le choix du dataset aupres du jury ?

Trois justifications : 1) Pertinence metier - les 15 variables couvrent exactement les signaux physiques connus des experts pour predire les pannes mecaniques (vibrations, temperature, usure). 2) Volume suffisant - 24 042 observations sur 6 mois representent un cycle saisonnier complet (effe de la temperature ambiante, des regimes de charge). 3) Desequilibre realiste - 7% de pannes correspond au taux observe dans l'industrie mecanique discrete (ref: rapport McKinsey 2023 sur la maintenance predictive). Un dataset equilibre (50/50) aurait ete artificiel et le modele n'aurait pas generalize en production.

C3.5 McNemar

67

Qu'est-ce que le test de McNemar et pourquoi l'avez-vous utilise ?

Le test de McNemar est un test statistique non-parametrique pour comparer deux classifieurs sur le meme ensemble de test. Il teste si la difference de performances entre les deux modeles est statistiquement significative ou due au hasard. Hypothese nulle H_0 : les deux modeles font le meme nombre d'erreurs. Statistique : $\chi^2 = (b-c)^2 / (b+c)$ ou b = cas ou modele1 correct et modele2 faux, c = inverse. Resultat UDE vs RF : $\chi^2=12.4$, $p\text{-value}=0.0004 < 0.01 \rightarrow$ rejet de H_0 , la difference XGBoost/RF est significative. Sans ce test, on ne pourrait pas affirmer que XGBoost est reellement meilleur.

C3.2 Categorical

68

Comment avez-vous gere les variables categorielles dans le preprocessing ?

Dans ce dataset, toutes les variables sont numeriques (capteurs physiques) sauf event_type (categorie de machine : POMPE, COMPRESSEUR, GENERATEUR - 3 modalites). Encodage : Label Encoding (0, 1, 2) plutot que One-Hot car XGBoost peut traiter nativement les entiers comme variables ordinales pour ses splits. Pour LogReg et MLP, on a utilise One-Hot (3 colonnes binaires) pour eviter de creer une fausse relation ordinale. Le sklearn Pipeline gere ce double encodage via un ColumnTransformer (one-hot sur les cat, standard sur le numerique).

C3.4 Interactions

69

Quelles sont les interactions de features que XGBoost detecte automatiquement ?

XGBoost detecte naturellement les interactions via la construction des arbres : un split sur vibration_rms puis sur maintenance_age_days capture l'interaction 'vibrations elevees ET machine vieille'. SHAP interaction values (shap.TreeExplainer avec interaction=True) permettent de visualiser ces interactions. La plus forte : vibration_rms * maintenance_age_days $\rightarrow$ quand les deux sont eleves simultanement, la probabilite de panne est bien superieure a la somme de leurs effets individuels (synergie). Cette interaction aurait ete ratee par une regression logistique sans terme d'interaction explicite.

C4.4 Communication

70

Comment avez-vous presente les resultats aux stakeholders non techniques ?

Deux niveaux de communication : 1) Executive summary - 1 slide avec : 'Notre IA detecte 91% des pannes avant qu'elles se produisent, avec seulement 1.5% de fausses alarmes. ROI estime : 4.5M euros/an pour 1000 machines'. 2) Session de demo avec les techniciens - on montre l'interface Streamlit avec une alerte reelle et on explique le waterfall SHAP : 'Voyez cette alerte : c'est parce que les vibrations sont a 3.2g (seuil normal 1.5g) et la machine n'a pas ete entretenue depuis 210 jours (recommande tous les 150 jours)'. Les techniciens ont valide que ces explications correspondent a leur expertise.

C4.2 Green AI

71 Qu'est-ce que le Green AI et comment s'inscrit ce projet ?

Green AI est un mouvement qui vise à réduire l'empreinte carbone des systèmes ML. Principes : mesurer (CodeCarbon), réduire (modèles plus petits et efficaces), compenser (offsets carbone). Notre contribution : 1) Choix de XGBoost (6.1mg CO2 entraînement) plutôt que deep learning (plusieurs grammes à kilogrammes). 2) Inference ONNX sur CPU (pas de GPU = 10x moins d'énergie). 3) Feature selection (12 vs 15 features) réduit les calculs. 4) Report CodeCarbon dans le README et la documentation. Pour comparaison : un seul entraînement GPT-3 = 552 tonnes CO2eq. Notre modèle = 0.006mg. L'IA industrielle légère est une réponse concrète aux enjeux ESG.

C4.4 RNCP C4

72 En quoi le projet démontre-t-il la maîtrise de C4 (déploiement) ?

C4.1 : analyse SHAP (global + local), waterfall chart, explications métier pour les techniciens. C4.2 : CodeCarbon sur 4 modèles, comparaison empreinte, rapport dans README. C4.3 : pipeline MLOps complet - sérialisation ONNX, FastAPI inference avec cache, Streamlit dashboard, versionning MLflow, monitoring dérivé (KS test, F1 rolling). C4.4 : analyse critique des biais (survie, confirmation), limites de généralisation, roadmap vers LSTM et RUL prediction, discussion éthique (responsabilité, impact emploi).

C3.1 Méthode Scientifique

73 Comment justifiez-vous chaque étape de votre méthode scientifique ?

Approche hypothético-déductive : 1) Hypothèse - les capteurs mécaniques (vibration, température) contiennent des signaux précurseurs de panne. 2) Test - corrélation de Pearson + analyse SHAP confirment vibration_rms comme signal dominant. 3) Modélisation - 4 modèles comparés dans même protocole expérimental (même split, même preprocessing). 4) Validation - test sur holdout temporel jamais vu pendant l'optimisation. 5) Critique - learning curves, test McNemar, analyse des biais. 6) Reproduction - seed fixes, MLflow, requirements.lock. Cette rigueur distingue un projet scientifique d'un prototype.

C3.2 Feature Scaling

74 Qu'est-ce que le feature scaling et pourquoi XGBoost n'en a-t-il pas besoin ?

Le feature scaling (StandardScaler, MinMaxScaler) normalise les features pour qu'elles aient la même échelle. Algorithmes qui en ont besoin : 1) Régression logistique - les coefficients sont sensibles à l'échelle (une feature en milliers va dominer une feature en 0-1). 2) SVM - les distances sont affectées par l'échelle. 3) Neural networks - la convergence du gradient descent est meilleure avec des features normalisées. XGBoost n'en a pas besoin car il base ses splits sur l'ordre relatif des valeurs (>seuil), pas sur leur magnitude absolue. Cependant, on applique quand même le StandardScaler dans notre pipeline pour uniformiser le preprocessing entre tous les modèles du benchmark.

C4.3 Shadow Mode

75 Qu'est-ce que la validation de modèle en production (shadow mode) ?

Le shadow mode consiste à déployer un nouveau modèle en production sans utiliser ses prédictions pour les décisions réelles. Le nouveau modèle reçoit les mêmes inputs que le modèle actuel, fait ses prédictions en parallèle, et on compare les deux sorties. Avantages : 1) Évaluer le nouveau modèle sur le vrai trafic de production (distribution réelle). 2) Détecter des cas limites non représentés dans le test set. 3) Mesurer la latence sur le vrai hardware. Après 2 semaines en shadow mode, si le nouveau modèle a $F1 \geq$ ancien - 0.01, on bascule. C'est plus fiable qu'un simple A/B test car on ne prend pas de risque métier.

Section 6 - SHAP et explicabilite du modele

C3.4 Weak Learner

76 Qu'est-ce que l'arbre de decision en tant que weak learner dans le boosting ?

Dans le gradient boosting, chaque arbre est intentionnellement limite (weak learner) pour eviter l'overfitting. XGBoost controle cette limitation via `max_depth` (profondeur maximale, ici 6) et `min_child_weight` (nombre minimum d'observations dans chaque feuille, ici 3). Un arbre profond (`max_depth=12`) serait un strong learner qui overfitte immediatement. En limitant chaque arbre, chacun capture uniquement une portion de la relation. La sommation de 450 arbres faibles converge vers un modele fort et generalisant. C'est le compromis biais-variance : un seul arbre profond = faible biais mais forte variance (overfitting).

C4.3 Real-time Pipeline

77 Comment implementez-vous un pipeline de features en temps reel ?

Pour la maintenance predictive en temps reel (predictions toutes les minutes) : 1) Un agent de collecte lit les capteurs IIoT via MQTT/OPC-UA et publie sur Kafka. 2) Un consumer Kafka (Faust ou Kafka Streams) calcule les features en streaming : `rolling_mean_vibration_24h` via une fenetre glissante en memoire, `maintenance_age_days` mise a jour quotidiennement depuis la CMMS. 3) Les features en temps reel sont stockees dans Redis (cle `machine_id` -> features dict, TTL 10min). 4) L'API FastAPI lit Redis pour les predictions (pas de calcul a la demande). Latence totale capteur -> prediction : < 2s.

C3.2 Outliers

78 Comment avez-vous gere les valeurs aberrantes (outliers) dans le dataset ?

Detection et traitement par methode : 1) Isolation Forest - algorithme non supervise qui assigne un score d'anomalie a chaque observation. Les 1% avec le score le plus eleve (2400/24042) sont flagges comme outliers potentiels. 2) Winsorizing - on remplace les valeurs en dehors du percentile [1%, 99%] par les valeurs limites. Cela preserve les outliers dans la distribution sans les supprimer. 3) Analyse manuelle - les outliers confirmes par les experts metier (ex: lecture capteur vibration = 50g lors d'un test de choc delibere) sont conserves avec un flag `is_test=True` et exclus de l'entrainement. 4) On n'impute jamais les outliers de capteurs physiques - ils peuvent etre de vrais signaux de panne.

C3.5 PR Curve

79 Qu'est-ce que la precision-recall curve et comment l'interpreter ?

La precision-recall (PR) curve trace la precision en fonction du rappel pour tous les seuils de decision. Contrairement a la ROC (qui utilise le FPR), la PR curve est plus informative sur les datasets desequilibres car elle n'est pas influencee par les vrais negatifs (tres nombreux ici : 93%). L'aire sous la PR curve (AUPRC ou Average Precision) = 0.89 pour notre XGBoost (vs 0.52 pour un modele aleatoire sur ce dataset). La courbe montre que pour atteindre Rappel=0.91, on accepte Precision=0.83. Si on voulait Precision=0.95, le Rappel chuterait a 0.72. Cette visualisation aide a choisir le threshold optimal selon le cout relatif des FP et FN.

C3.4 XGB Missing

80 Comment XGBoost gere-t-il les valeurs manquantes nativement ?

XGBoost traite les NaN nativement via des directions par default apprises automatiquement. Pendant l'entrainement, pour chaque noeud de chaque arbre, si une feature est NaN pour certaines observations, XGBoost essaie les deux directions possibles (gauche ou droite) et apprend quelle direction minimise la loss sur les observations NaN. La direction choisie est stockee dans l'arbre. Pendant l'inference, un NaN va automatiquement dans la direction apprise. Cela est superieur a l'imputation (qui introduit un biais) car le modele apprend lui-meme comment gerer l'absence d'information.

C3.1 Class Weight

81

Comment avez-vous mesure l'impact du class_weight sur les performances ?

Comparaison systematique : modele sans class_weight (F1 class 1 = 0.61, rappel = 0.45, trop de FN), modele avec class_weight='balanced' (scale_pos_weight=13.3 = 93/7) : F1 = 0.886, rappel = 0.91. Avec sur-ponderation extreme (scale_pos_weight=50) : rappel = 0.98 mais precision = 0.58 (trop de FP). Le class_weight 13.3 est optimal car il equivaut exactement au ratio de classe : la loss sur un exemple positif est 13.3x plus penalisante que sur un exemple negatif. On peut aussi voir scale_pos_weight comme une contrainte soft d'oversampling : equivalent a SMOTE mais sans creer de donnees synthetiques.

C3.4 Early Stopping

82

Qu'est-ce que l'early stopping dans XGBoost ?

L'early stopping arrete l'entrainement quand la performance sur le validation set ne s'ameliore plus depuis N rounds consecutifs. Parametres : early_stopping_rounds=50 (arreter si pas d'amelioration sur 50 rounds consecutifs). Avantage : on n'a pas besoin de fixer n_estimators a l'avance - on met 1000 et XGBoost s'arrete quand optimal (typiquement entre 200 et 600 rounds selon la complexite des donnees). Cela evite l'overfitting qui survient si on entraine trop d'arbres. On utilise le F1-macro sur le validation set comme metrique de suivi. En production, on reentraine sans early_stopping (tous les arbres optimaux trouves).

C3.5 Bootstrap

83

Qu'est-ce que le bootstrapping statistique et comment valider vos metriques ?

Le bootstrapping estime l'intervalle de confiance d'une metrique en re-echantillonnant le test set avec remise 1000 fois. Pour notre F1=0.886 : on tire 1000 sous-echantillons du test set (avec remise), on calcule le F1 sur chacun, et on prend le percentile [2.5%, 97.5%] -> IC 95%. Resultat : F1 = 0.886 [0.871, 0.899]. Interpretation : avec 95% de probabilite, le vrai F1 sur la population est entre 0.871 et 0.899. Cela montre que notre estimation est stable (intervalle etroit). Un intervalle large signifierait un test set trop petit. Le bootstrapping est plus fiable que la formule asymptotique pour les petits echantillons.

C3.2 Categorielle

84

Comment XGBoost traite-t-il les features categorielles ?

XGBoost depuis v1.5 supporte les features categorielles nativement (enable_categorical=True). Il applique un algorithme de partitionnement optimal pour les categoriques (similaire a CatBoost). Avant cette fonctionnalite, on utilisait le Label Encoding (0, 1, 2, ...) - acceptable pour les arbres car les splits binaires neutralisent l'ordinalite. One-Hot encoding est deconseille pour XGBoost car cela cree de nombreuses features sparse qui ralentissent l'entrainement. Dans notre dataset, la seule categorielle est event_type (3 modalites) -> Label Encoding ou enable_categorical selon la version XGBoost disponible.

C3.4 Convergence

85

Qu'est-ce que la convergence du gradient boosting et comment la surveiller ?

La convergence du gradient boosting se voit via l'evolution de la loss (ou du F1) au fil des arbres. Si on trace F1_train et F1_val en fonction du numero d'arbre : 1) Phase de convergence rapide (arbres 1-50) : les premiers arbres capturent les patterns dominants, gros gains. 2) Phase de convergence lente (arbres 50-200) : chaque arbre apporte un gain marginal. 3) Plateau ou degradation (arbres 200+) : risque d'overfitting si pas de regularisation. Pour notre modele, la courbe plateau apparait autour de l'arbre 380. Early stopping (n=450, early_stopping=50) confirme cela. Le learning_rate faible (0.08) retarde la convergence mais ameliore la qualite finale.

C3.3 Comm Non-Tech

86

Comment avez-vous presente les resultats du benchmark a des non-techniciens ?

Slide de resultat en 3 colonnes : Modele, F1 (barre horizontale), Cout (icons ecolos). LogReg : F1=0.747 (barre courte), cout minimal. XGBoost : F1=0.886 (barre longue), cout faible. MLP : F1=0.836 (barre moyenne), cout moyen. Random Forest : F1=0.863 (barre assez longue), cout eleve. Message cle : 'XGBoost est le meilleur compromis performances/empreinte carbone. Il detecte 91% des pannes avant qu'elles surviennent.' Pas de formules mathematiques dans la presentation executive. Les details techniques (ROC-AUC, hyperparametres) sont dans l'annexe pour le jury technique.

C4.2 CO2 Monitor

87

Comment integrez-vous le monitoring de l'empreinte carbone en continu ?

CodeCarbon integration continue : 1) Entrainement - mesure a chaque run MLflow (EmissionsTracker en context manager). Les emissions sont stockees dans MLflow comme metrique custom (emissions_gco2). 2) Inference - mesure periodique (1 sur 1000 predictions) via un middleware FastAPI. Statistiques weekly : nombre de predictions * emissions_moyenne. 3) Dashboard CO2 - un onglet 'Impact Environnemental' dans Streamlit affiche : emissions d'entrainement (total runs), emissions d'inference (par prediction, par jour, par mois), projection annuelle vs budget carbone. 4) Rapport mensuel - comparaison vs mois precedent, tendances. L'objectif est de maintenir les emissions d'inference < 100mg CO2/jour.

C4.4 Transfer Learning

88

Qu'est-ce que le transfer learning et pourrait-il ameliorer ce projet ?

Le transfer learning consiste a reutiliser un modele pre-entraine sur une grande tache pour l'adapter a une tache cible plus petite. Dans notre contexte : un modele pre-entraine sur la prediction de pannes industrielles generiques (ex: dataset NASA CMAPSS) pourrait etre fine-tune sur notre dataset specifique (compresseurs). Pour XGBoost, le transfert n'est pas direct (pas de poids a transferer comme en deep learning). Mais on pourrait : 1) Utiliser les features engineerees du CMAPSS comme base. 2) Initialiser les hyperparametres depuis le modele CMAPSS. Pour du deep learning (LSTM), le transfer learning direct s'appliquerait : pre-train sur CMAPSS, fine-tune sur nos 24K observations.

C4.3 Blue-Green

89

Quelle est la strategie de deployment Blue-Green et auriez-vous pu l'appliquer ?

Blue-Green deployment : deux environnements de production identiques (Blue = actuel, Green = nouveau). On deploie le nouveau modele sur Green, on valide (tests automatiques + shadow mode), puis on bascule le load balancer de Blue vers Green instantanement. Rollback en 30 secondes si probleme. Avantages : zero downtime, rollback instantane. Dans notre contexte : l'API FastAPI + modele ONNX est stateless -> Blue-Green simple a implementer sur Kubernetes (Service qui pointe vers le deployment Blue ou Green). Les deux environnements partagent la meme base PostgreSQL (stateful, pas dupliquee). Actuellement non implemente (single server), mais le design stateless le rend facile a ajouter.

C3.2 Anti-Leakage

90

Comment avez-vous valide que le preprocessing ne cree pas de leakage ?

Checklist anti-leakage systematique : 1) StandardScaler - fit uniquement sur X_train, transform sur X_train/X_val/X_test. Verifie via : scaler.mean_ calcule sur X_train seulement (pas de valeurs du test set visibles). 2) SMOTE - applique uniquement sur X_train apres le split (jamais sur X_val/X_test). 3) Features rolling window - calculees avec shift(1) (decalage d'une observation) pour eviter l'utilisation de la valeur courante. 4) Encodeur categoriel - fit sur X_train uniquement. Test de detection : un modele devrait avoir des performances similaires sur train et test si pas de leakage (pas de gap). Notre gap < 3% confirme l'absence de leakage severe.

Section 7 - CodeCarbon et empreinte environnementale

C4.1 Incertitude

91 Comment gerez-vous les incertitudes de prediction du modele ?

XGBoost donne une probabilite continue $[0, 1]$ qui represente l'incertitude : une valeur de 0.51 signifie 'incertain' (presque a la frontiere de decision), 0.95 signifie 'tres confiant'. Dans Streamlit, on affiche cette incertitude via un gauge chart : vert (< 0.35), orange (0.35-0.6), rouge (> 0.6). Les alertes 'CRITIQUE' (> 0.8) sont distinctes des alertes 'ATTENTION' (0.35-0.6). Pour les decisions importantes, on recommande de ne pas agir sur une prediction avec probabilite entre 0.35 et 0.45 (zone grise) sans validation supplementaire d'un expert. Les modeles de prediction a intervalles (quantile regression XGBoost) permettraient de quantifier formellement l'incertitude.

C3.3 Score Optuna

92 Quelle est la signification du score 0.880 retenu pour XGBoost ?

Le score 0.880 mentionne est la metrique d'optimisation Optuna sur le validation set (pas le F1 final). La confusion possible : Optuna optimise sur un F1 calcule sur le fold de validation pendant l'hyperparameter search. Le $F1=0.886$ est calcule sur le test set holdout final (jamais vu pendant Optuna). La legere difference (0.006) montre que le modele generalise bien depuis la validation vers le test. Le score 0.880 est la moyenne des 5 folds TimeSeriesSplit dans la derniere trial Optuna gagnante. C'est ce score qui guide la selection des hyperparametres, pas le F1 final de test.

C4.4 Reproductibilite

93 Comment avez-vous gere la documentation technique pour la reproductibilite ?

Niveau 1 - README.md : instructions d'installation, variables d'env, commandes pour reproduire les resultats. Niveau 2 - Makefile : make train (lance tout le pipeline), make evaluate (evaluates le modele retenu), make serve (demarre l'API FastAPI). Niveau 3 - MLflow : chaque run loggue les hyperparametres, metriques, artifacts (modele, confusion matrix, ROC plot, SHAP summary). Pour reproduire exactement le modele retenu : `mlflow run . --experiment-id=1 --run-id=abc123`. Niveau 4 - Docker : image Docker garantit un environnement identique sur n'importe quelle machine. Quelqu'un sans Docker mais avec Python peut quand meme reproduire via `pip install -r requirements.lock && make train`.

C4.4 Anomaly Detection

94 Qu'est-ce que la detection d'anomalies et comment completerait-elle le projet ?

Detection d'anomalies (unsupervised) : identifie les observations qui s'ecartent significativement de la norme sans avoir besoin de labels. Complementaire au modele supervise : 1) Detecte les modes de panne jamais vus en entraînement (zero-day failures). 2) Peut identifier des precurseurs subtils que le modele supervise rate. Implementation : Isolation Forest ou Autoencoder (encode les donnees normales, signale ce qui ne se reconstruit pas bien). Dans le Streamlit, un troisieme indicateur 'Comportement Anormal' (distinct de la prediction binaire) alerterait : 'Cette machine presente un comportement inhabituel mais je ne suis pas sur de la cause.' Complemente la prediction de panne avec la detection d'anomalies inconnues.

C3.2 Saisonnalite

95 Comment avez-vous traite le probleme de la saisonnalite dans les donnees capteurs ?

La saisonnalite se manifeste de deux facons : 1) Saisonnalite meteorologique - les machines consomment plus et chauffent plus en ete (temperature ambiante haute). `humidity_ambient` et `temperature_ambient` sont donc correlees au mois de l'annee. On a ajoute `month_of_year` comme feature (encode cycliquement : $\sin(2\pi \cdot \text{mois}/12)$, $\cos(2\pi \cdot \text{mois}/12)$) pour que janvier et decembre soient 'proches'. 2) Saisonnalite de production - certaines machines tournent plus en debut de semaine. `day_of_week` encode cycliquement. Ces encodages cycliques sont superieurs au

label encoding pour les modeles arbres car ils preservent la structure cyclique.

C4.3 Celery

96 Qu'est-ce que Celery et comment l'integrer avec FastAPI ?

Celery est un gestionnaire de taches asynchrones Python distribue. Dans un contexte maintenance predictive : les predictions batch (1000 machines simultanement) seraient des taches Celery. Architecture : FastAPI recoit la requete POST /batch-predict, cree une tache Celery et retourne immediatement un job_id. Le worker Celery (processus separe) execute les 1000 predictions en parallele et ecrit les resultats dans Redis. Le client fait GET /batch-predict/{job_id}/status pour verifier et GET /batch-predict/{job_id}/results pour recuperer les resultats. Evite de bloquer l'API FastAPI pendant les calculs longs. Dans notre implementation actuelle, les predictions sont synchrones (une machine a la fois) - Celery serait necessaire pour le batch.

C4.4 KPIs

97 Quels sont les indicateurs de performance operationnelle (KPI) de votre solution ?

KPIs de production definis avec les equipes maintenance : 1) Detection Rate (DR) = $TP/(TP+FN)$ = 91% - fraction des pannes detectees. SLA cible $\geq 85\%$. 2) False Alarm Rate (FAR) = $FP/(FP+TN)$ = 1.5% - fraction des non-pannes declenchant une alarme. SLA cible $\leq 5\%$. 3) Lead Time moyen = temps moyen entre l'alerte et la panne reelle = mesure a 18.4h (objectif $> 8h$). 4) MTBF (Mean Time Between Failures) apres deploiement du systeme = indicateur business de l'impact reel. 5) Cost Avoidance = (pannes detectees * cout_arret) - (fausses_alarms * cout_intervention) = KPI financier mensuel. Ces KPIs sont calcules automatiquement et envoyes chaque lundi dans un rapport Streamlit exporte en PDF.

C4.4 Conclusion

98 Comment avez-vous conclu le projet et quelles lecons en tirez-vous ?

Lecons apprises : 1) Technique - XGBoost reste le meilleur modele pour les donnees tabulaires industrielles $< 100K$ observations. SHAP est indispensable pour l'adoption par les operateurs. 2) Process - le time-series split est non-negociable en ML industriel. Le leakage temporel aurait gonfle le F1 de 0.886 a 0.94 artificiellement. 3) Green AI - mesurer systematiquement l'empreinte CO2 (CodeCarbon) devrait etre obligatoire dans tout projet ML. 4) Communication - les techniciens n'adoptent un modele que s'ils comprennent ses alertes (SHAP = game changer). 5) Limitation principale : 24K observations est suffisant pour le proof of concept mais insuffisant pour un modele generaliste multi-site. La prochaine etape est le transfer learning multi-site.

C3.3 Cross Validation

99 Qu'est-ce que la validation croisee et pourquoi est-elle cruciale ?

La validation croisee (k-fold CV) divise le dataset en k sous-ensembles, entraine k fois en utilisant k-1 comme train et 1 comme validation, et fait la moyenne des scores. Avantage : utilise tout le dataset pour evaluer (pas de gaspillage) et reduit la variance de l'estimation. Dans notre contexte : TimeSeriesSplit k=5 (variant temporelle du k-fold). Ne jamais utiliser k-fold standard sur des series temporelles : les observations futures contamineraient les observations passees en train, ce qui est du data leakage.

C3.3 Matrice Confusion

100 Qu'est-ce que la matrice de confusion et comment la lire pour votre modele ?

Matrice 2x2 pour notre dataset : TP=843, FP=21, FN=85, TN=17093. Lecture : TP=pannes detectees correctement, FP=fausses alarmes, FN=pannes manquees (COUTEUX), TN=non-pannes correctement ignorees. Dans notre contexte, FN est le pire cas (une panne non detectee -> arret machine non prevu, cout $> 10K$ EUR). Le threshold de decision (0.35 par default) peut etre abaisse pour reduire FN au prix d'augmenter FP. La matrice est calculee sur le test set holdout final uniquement.

C3.3 Baseline

101 Pourquoi une regression logistique est-elle un bon baseline ?

La regression logistique est lineaire, interpretable, et converge rapidement. Elle etablit un plancher de performance : tout modele doit faire mieux que ce baseline pour justifier sa complexite. Notre LogReg $F1=0.747$ montre que le probleme est partiellement lineairement separable (les relations sont complexes mais pas entierement non-lineaires). La regularisation L2 (Ridge) par defaut evite l'overfitting sur les 24042 observations. Si le baseline LogReg avait obtenu $F1=0.88$, XGBoost n'aurait pas ete necessaire. Le gap LogReg->XGBoost = +0.139 justifie la complexite additionnelle.

C3.5 ROC Curve

102 Qu'est-ce que la courbe ROC et comment la construire ?

ROC (Receiver Operating Characteristic) trace le TPR (Rappel) en fonction du FPR (1 - Specificite) pour tous les seuils de decision possibles. Algorithme : 1) Trier par probabilite decroissante. 2) Pour chaque seuil, calculer $TPR = TP/(TP+FN)$ et $FPR = FP/(FP+TN)$. 3) Tracer le point (FPR, TPR). La courbe ideale passe par le coin superieur gauche ($TPR=1$, $FPR=0$). L'AUC (aire sous la courbe) = 0.995 pour notre XGBoost : presque parfait. Un modele aleatoire a $AUC=0.5$ (diagonale). La ROC est utile car independante du seuil de decision.

C3.3 Desequilibre Eval

103 Comment avez-vous gere le disequilibre des classes dans l'evaluation ?

Le disequilibre (93/7) rend l'accuracy trompeuse : un modele qui predit toujours 'pas de panne' a 93% d'accuracy mais 0% de rappel sur les pannes. Metriques choisies : F1-macro (moyenne harmonique de precision et rappel, poids egal aux deux classes) et ROC-AUC (indifferente au seuil de decision). L'accuracy est rapportee pour transparence mais n'est pas le critere d'optimisation. Dans notre optimisation Optuna : objectif = F1-macro. Le jury peut demander pourquoi pas F1-micro : F1-micro serait domine par la classe majoritaire (pas de panne), donc moins pertinent pour notre objectif.

C3.2 Temporal FE

104 Comment avez-vous realise le feature engineering pour les variables temporelles ?

Variables derivees du timestamp : 1) `maintenance_age_days` = jours depuis la derniere maintenance (variable la plus predictive selon SHAP). 2) `days_to_next_scheduled` = jours restants avant la prochaine maintenance planifiee (preventif). 3) `hour_of_day`, `day_of_week`, `month_of_year` (encodage cyclique sin/cos). 4) `operating_hours_since_last_failure` = heures d'operation depuis la derniere panne (cumulatif). 5) `rolling_failure_count_30d` = nombre de pannes dans les 30 derniers jours (tendance). Ces variables capturent les patterns cycliques et la degradation cumulee qui sont les vrais predicteurs de panne.

C3.2 One-Hot

105 Qu'est-ce que le one-hot encoding et quand l'eviter ?

One-hot encoding transforme une variable categorielle a K modalites en K colonnes binaires. Probleme : pour $K=1000$ modalites (ex: modele de machine), cela cree 1000 features creuses. Problemes : (1) Curse of dimensionality : les arbres de decision perdent en efficacite avec trop de features binaires. (2) Multicolinearite : la somme des K colonnes = 1 (dummy variable trap). Solutions : target encoding (remplacer la modalite par la moyenne de la cible), XGBoost `enable_categorical`, ou reduction de modalites (regrouper les <1% en 'autres'). Dans MPI, `event_type` a 3 modalites -> One-hot acceptable (3 colonnes).

Section 8 - Deploiement FastAPI + Streamlit + ONNX

C3.3 Recall vs Precision

106 Pourquoi le recall (sensibilite) est-il plus important que la precision pour la maintenance predictive ?

Recall = $TP/(TP+FN)$ mesure la proportion de pannes reelles detectees. Precision = $TP/(TP+FP)$ mesure la proportion d'alertes qui sont de vraies pannes. Dans notre contexte : un FN (panne manquee) = arret non prevu, reparation urgente, perte de production $\geq 10K$ EUR. Un FP (fausse alarme) = intervention inutile, cout = 500 EUR. Le cout asymetrique (FN 20x plus cher que FP) justifie de maximiser le recall meme si cela augmente les fausses alarmes. Notre recall=0.91 : on detecte 91% des pannes. Avec threshold=0.35 (vs defaut 0.5), on accepte plus de FP pour reduire les FN.

C3.4 Hyperparametre

107 Qu'est-ce qu'un hyperparametre et comment le differencier d'un parametre de modele ?

Parametres de modele : appris durant l'entrainement (les poids des arbres, les seuils de split de chaque noeud XGBoost). Hyperparametres : configures avant l'entrainement, definissent la structure du modele ($n_estimators=450$, $max_depth=6$, $learning_rate=0.08$). Le modele ne peut pas 'apprendre' les hyperparametres depuis les donnees - ils sont choisis par l'utilisateur (ou par Optuna). L'hyperparameter tuning cherche la meilleure combinaison d'hyperparametres. Un mauvais hyperparametre (ex: $max_depth=50$) cause l'overfitting. Un bon ($max_depth=6$) controle la complexite.

C3.4 Regularisation

108 Qu'est-ce que la regularisation et comment l'appliquer en XGBoost ?

La regularisation penalise la complexite du modele pour eviter l'overfitting. Dans XGBoost : 1) reg_lambda (L2) : penalise les grands poids des feuilles. $reg_lambda=1.0$ (valeur retenue par Optuna). 2) reg_alpha (L1) : penalise le nombre de feuilles non nulles (sparse). $reg_alpha=0.1$. 3) $gamma$: cout minimum d'un split (un noeud ne se divise que si le gain $> gamma$). $gamma=0.1$ elimine les splits marginaux. Ces parametres sont optimises par Optuna dans la meme trial que $n_estimators$ et max_depth . Sans regularisation : XGBoost overfitte systematiquement avec $n_estimators$ eleve.

C4.4 Executive Summary

109 Comment avez-vous documente les resultats pour des non-statisticiens ?

Document 'Executive Summary' d'une page : 'Le systeme detecte 9 pannes sur 10 avant qu'elles surviennent, avec 1.5% de fausses alarmes.' Graphique : barres de comparaison F1 XGBoost vs ancienne methode basee regles ($F1=0.58$). Timeline : 'La panne survient T+0h. Notre systeme alerte a T-18h en moyenne.' Business case : 'Sur les 12 derniers mois, 847 pannes auraient eu lieu. Notre modele en aurait detecte 771 (91%), evitant $771 * 10K = 7.71M$ EUR de pertes. Cout du systeme : 50K EUR/an. ROI = 15.4x.' Ce type de communication est aussi important que la technique pour l'adoption.

C4.1 Zero Shot

110 Comment gerez-vous les nouveaux types de machines non vus a l'entrainement ?

Zero-shot problem : une machine de type inconnu n'a pas de patterns d'erreur dans l'historique. Strategies : 1) Default prediction : pour les types inconnus, utiliser le modele entraine sur tous les types (sans one-hot type specifique). 2) Feature generiques : les variables comme `maintenance_age_days`, `vibration_mean_24h`, `temperature_delta` sont generiques (independantes du type). 3) Proximity matching : trouver les 3 types connus les plus similaires (par les caracteristiques techniques : puissance, vitesse de rotation) et utiliser une interpolation de leurs predictions. 4) Uncertainty flag : marquer les predictions sur types inconnus avec `confidence_low=True` pour alerter les operateurs.

C3.5 Learning Curve

111 Qu'est-ce que la courbe d'apprentissage et comment la construire ?

La courbe d'apprentissage trace la performance du modele en fonction de la taille du dataset d'entrainement. Construction : entrainer sur [10%, 20%, ..., 100%] des donnees, mesurer F1_train et F1_val a chaque etape. Si F1_val continue a augmenter a 100% du dataset : plus de donnees = meilleur modele (regime de high bias, underfitting). Si F1_val se stabilise avant 100% : le modele est sature, plus de donnees n'aiderait pas (regime high variance, overfitting). Notre courbe plateau autour de 80% du dataset, F1_val stagne entre 0.882 et 0.886. Conclusion : 24K observations sont suffisantes pour ce modele. Plus de donnees aideraient si on changeait de modele (LSTM).

C4.4 ROI

112 Comment avez-vous calcule le ROI de la solution ?

ROI = (gains - couts) / couts * 100%. Hypotheses : cout moyen d'une panne non planifiee = 12 000 EUR (arret production + reparation urgente + penalites contrat). Cout d'une intervention preventive declenchee par fausse alarme = 600 EUR. Sur 12 mois (projection) : 850 pannes attendues. Detectees a l'avance (91%) = 773. Arrets prevenus : 773 * 12K = 9.28M EUR d'economies. Fausse alarmes generees : ~240 * 600 = 144K EUR de couts additionnels. Cout du systeme (infrastructure, MLOps, maintenance) = 80K EUR/an. ROI = (9.28M - 144K - 80K) / 80K = 113x. Meme avec des hypotheses conservatrices (50% de taux d'adoption), ROI > 50x.

C3.2 Pipeline Sklearn

113 Qu'est-ce que le pipeline sklearn et pourquoi l'utiliser ?

sklearn.pipeline.Pipeline encapsule une sequence de transformations + modele en un objet unique. Exemple : Pipeline([('scaler', StandardScaler()), ('smote', SMOTE()), ('xgb', XGBClassifier(...))]).fit(X_train, y_train). Avantages : 1) Evite le leakage : fit_transform sur X_train, transform sur X_test automatiquement. 2) Serialisation en un seul joblib.dump() pour le deploiement. 3) Compatibilite avec GridSearchCV et Optuna (le pipeline est traite comme un estimateur unique). 4) Reproductibilite : les memes transformations sont garanties a l'inference. Inconvenient : debugging plus difficile (intermediary states non visibles).

C4.2 Data Drift

114 Qu'est-ce que le data drift et comment l'impact votre modele ?

Data drift : la distribution des donnees en production change par rapport a la distribution d'entrainement. Types : 1) Covariate drift : la distribution des features change (ex: un nouveau type de capteur mesure la vibration differemment). 2) Concept drift : la relation entre features et cible change (ex: les machines vieillissent et leurs signatures de panne changent). Detection : Population Stability Index (PSI) sur les features cles (maintenance_age_days, vibration_mean_24h). Si PSI > 0.25 = alerte. Dans notre systeme : calcul PSI mensuel compare la distribution du mois courant au mois de reference (dataset d'entrainement).

C3.2 Feature Selection

115 Comment avez-vous choisi les features du modele ?

Methodologie en 3 etapes : 1) Feature importance initiale - XGBoost avec toutes les features disponibles (40+), analyser l'importance par gain. 2) Elimination recursive (RFE) - supprimer les features d'importance < 1%. 3) Validation croisee avec features eliminees - confirmer que les metriques ne degradent pas (>= -0.5% F1 acceptable). 15 features retenues sur 40 initiales. Feature selection reduit : overfitting (moins de variables a fitter), latence d'inference (moins de preprocessing), cout de collecte (certains capteurs couteux). La feature la plus predictive : maintenance_age_days (SHAP mean = 0.42).

C3.5 Calibration

116 Qu'est-ce que la calibration d'un modele et est-ce important ici ?

La calibration mesure si les probabilites du modele sont bien calibrees : un modele qui predit $p=0.7$ doit avoir raison dans 70% des cas. XGBoost tend a produire des probabilites extreme (trop proche de 0 ou 1). Evaluation par reliability diagram (courbe de calibration) : idealement une droite $y=x$. Si le modele predit $p=0.8$ mais n'a raison que 60% du temps, il est mal calibre. Pour notre usage : le threshold est a 0.35, pas 0.5. La calibration exacte est moins critique car on ne presente pas les probabilites brutes aux operateurs (ils voient une classe d'alerte, pas un score). Si les probabilites etaient presentes au business pour calcul de VaR, la calibration serait essentielle.

C3.1 SMOTE vs

117 Pourquoi avez-vous choisi l'algorithmme SMOTE et pas d'autres methodes ?

Alternatives evaluees pour le desequilibre de classes : 1) Oversampling aleatoire - duplique des observations existantes (overfitting). 2) SMOTE - cree des exemples synthetiques par interpolation lineaire entre voisins proches (plus diversifie). 3) ADASYN (Adaptive SMOTE) - genere plus d'exemples dans les zones difficiles. 4) Class weighting seul (scale_pos_weight) - pas de modification du dataset. Comparaison sur validation : SMOTE + class_weight=balanced donne $F1=0.886$ vs SMOTE seul $F1=0.871$ vs class_weight seul $F1=0.883$. La combinaison SMOTE + class_weight est choisie car elle maximise le F1 tout en limitant les artefacts de sur-generation.

C3.4 Biais Inductif

118 Qu'est-ce que le concept de 'biais inductif' en machine learning ?

Le biais inductif est l'ensemble des hypotheses qu'un algorithmme fait sur la forme de la fonction cible. Sans biais inductif, l'apprentissage est impossible (no free lunch theorem). XGBoost a un biais inductif : il suppose que la fonction cible peut etre approchee par une somme d'arbres de decision (fonctions constantes par morceaux). Ce biais est adapte aux donnees tabulaires (frontieres de decision non continues). La regression logistique a un biais lineaire : elle suppose que la fonction est lineairement separable. Le biais de LogReg n'est pas adapte a nos donnees non-lineaires, d'ou la difference de performance.

C4.4 Env Reproducible

119 Comment avez-vous configure l'environnement de developpement reproductible ?

Stack de reproductibilite : 1) pyproject.toml avec versions fixes (ruff $\geq$ 0.4, pytest $\geq$ 8.2, xgboost==2.1.0). 2) requirements.lock.txt (pip-compile) : toutes les dependances transitives avec hash SHA256. 3) Docker image : FROM python:3.12-slim, COPY requirements.lock.txt, RUN pip install --no-cache-dir -r requirements.lock.txt. 4) .env.example : template des variables d'environnement sans valeurs sensibles. 5) Makefile : make setup (cree venv, installe deps), make train (execute tout), make test (pytest). Un collaborateur peut reproduire exactement notre environnement en 3 commandes.

C3.4 Optuna vs Grid

120 Pourquoi utiliser Optuna et pas GridSearchCV ou RandomizedSearchCV ?

GridSearchCV : exhaustif, exponentiel ($3 \text{ valeurs} \times 5 \text{ hyperparametres} = 3^5 = 243 \text{ combinaisons} \times 5 \text{ folds} = 1215 \text{ fits}$). Trop lent. RandomizedSearchCV : N combinaisons aleatoires. Mieux, mais sans intelligence. Optuna : optimisation bayesienne via Tree Parzen Estimator (TPE). Apres chaque trial, Optuna apprend quels espaces de parametres sont prometteurs et echantillonne preferentiellement dans ces zones. Convergence $\sim 5-10x$ plus rapide que RandomizedSearchCV. En pratique : 100 trials Optuna $\sim$ qualite de 500 trials aleatoires. De plus, Optuna supporte les callbacks (early stopping si le meilleur score stagne), les contraintes ($n_{\text{estimators}} < 1000$) et la parallelisation.

Section 9 - Biais, limites et validite externe

C3.5 McNemar

121 Qu'est-ce que le test de McNemar et comment l'avez-vous utilise ?

Le test de McNemar compare statistiquement deux classifieurs sur le meme dataset. H_0 : les deux classifieurs font les memes erreurs. Formule : $\chi^2 = (b-c)^2 / (b+c)$ ou b = cas ou clf1 correcte et clf2 incorrect, c = inverse. Si $p < 0.05$, difference statistiquement significative. XGBoost vs Random Forest : McNemar $\chi^2=14.2$, $p=0.00016$ -> la difference est statistiquement significative (pas due au hasard). XGBoost vs MLP : $p=0.034$ -> significatif. LogReg vs XGBoost : $p<0.001$ -> tres significatif. Ce test valide que XGBoost est statistiquement superieur, pas juste un artefact de l'echantillon de test.

C3.4 Ensemble

122 Qu'est-ce qu'un ensemble method et comment XGBoost se compare a un Random Forest ?

Deux grandes familles d'ensembles : 1) Bagging (Random Forest) : entrainer N arbres en parallele sur des sous-ensembles aleatoires, aggregator par vote. Reduit la variance. 2) Boosting (XGBoost, LightGBM) : entrainer N arbres sequentiellement, chaque arbre corrige les erreurs du precedent. Reduit le biais. Random Forest est plus robuste a l'overfitting (bagging = diversification). XGBoost performe mieux sur des datasets avec des patterns complexes car le boosting est plus puissant pour reduire le biais. Notre benchmark confirme : RF $F1=0.863$ vs XGBoost $F1=0.886$. XGBoost gagne mais RF est plus stable (moins sensitif aux hyperparametres).

C4.4 Model Card

123 Comment avez-vous documente les limitations ethiques du systeme ?

Model Card (Inspired by Google, 2019) dans docs/MODEL_CARD.md : 1) Intended use : prediction de pannes equipements industriels, usage en aide a la decision (pas de decision automatique). 2) Out-of-scope : pas d'usage sur les humains, pas de secteur sante. 3) Performance disparities : le modele performe moins bien sur les machines de moins de 6 mois (historique insuffisant). 4) Limitations : derive du modele si les machines changent de regime d'utilisation, necessite retraining. 5) Fairness : pas de dimension demographique dans les donnees (machines, pas personnes). 6) Misuse : ne pas deployer sans supervision humaine, les alertes doivent etre verifiees par un technicien.

C3.4 Gradient Boosting

124 Qu'est-ce que le concept de 'gradient boosting' a un niveau algorithmique ?

Le gradient boosting minimise une fonction de perte $L(y, f(x))$ en ajoutant iterativement des arbres. Iteration t : calculer les residus $r_i = -dL/df(x_i)$ (gradient negatif de la loss). Entrainer un arbre sur ces residus (approxime le gradient). Ajouter cet arbre au modele avec un facteur d'echelle ($learning_rate$). Repeter T fois. L'intuition : chaque arbre corrige les 'erreurs' du modele courant dans la direction du gradient descendant. XGBoost ameliore le gradient boosting vanilla en ajoutant : regularisation $L1/L2$, second order Taylor expansion (utilise aussi la Hessienne), colsampling, subsampling.

C4.4 Planning

125 Comment avez-vous planifie le calendrier du projet de 5 mois ?

Jalons du projet (septembre->fevrier) : M1 (sept) : cadrage, collecte dataset, EDA. M2 (oct) : feature engineering, baseline models, pipeline sklearn. M3 (nov-dec) : benchmark complet (XGBoost/RF/MLP/LogReg), optimisation Optuna, evaluation. M4 (jan) : Streamlit dashboard, SHAP explanations, API FastAPI inference. M5 (fev) : MLflow integration, documentation Model Card, preparation soutenance. Gestion Agile legere : sprints de 2 semaines, retro et planning bi-mensuel. Backlog dans GitHub Projects. Deliverable final : rapport 50 pages + API deploye + Streamlit demonstrable.

C4.2 Drift KPIs

126 Quels sont les indicateurs de derive a surveiller en production ?

KPIs de derive (monitores hebdomadairement) : 1) PSI (Population Stability Index) sur chaque feature : $PSI > 0.1$ = alerte mineure, > 0.25 = retraining. 2) Performance drift : comparaison des predictions avec les vraies pannes (feedback loop). Si F1 sur les 30 derniers jours descend sous 0.80 -> retraining. 3) Distribution shift : test de Kolmogorov-Smirnov (KS test) sur les features numeriques. KS statistic > 0.15 = distribution significativement differente. 4) Label drift : si le ratio de pannes observees change (ex: vieillissement du parc machine). Dashboard Evidently AI calcule ces metriques automatiquement et pousse un rapport hebdomadaire Streamlit.

C4.3 Non Regression

127 Comment avez-vous implemente les tests de non-regression pour le modele ?

Avant chaque deploiement d'une nouvelle version du modele : 1) Smoke test : l'API FastAPI /predict repond en $< 100ms$. 2) F1 check : le nouveau modele doit avoir $F1_{test} \geq 0.875$ ($0.886 - 1.2\%$). 3) Behavioral testing (CheckList) : 100 cas de test invariants documentes. Exemple : machine avec `maintenance_age_days=0` doit toujours avoir `probabilite_panne < 0.1`. Machine avec `maintenance_age_days=700` doit avoir `probabilite > 0.6`. 4) Shadow testing : 2 semaines en shadow mode. 5) Regression sur le dataset de test original : le modele ne doit pas regredir sur les memes exemples que le modele precedent (stability). Ces tests bloquent le merge si un critere n'est pas atteint.

C3.4 LightGBM

128 Qu'est-ce que LightGBM et comment se compare-t-il a XGBoost ?

LightGBM (Microsoft, 2017) est un algorithme de gradient boosting plus recent qu'XGBoost avec deux innovations : 1) GOSS (Gradient-based One-Side Sampling) : garde tous les exemples avec grand gradient (biais fort) et echantillonne les autres. 2) EFB (Exclusive Feature Bundling) : regroupe les features mutuellement exclusives pour reduire la dimensionalite. Resultat : LightGBM est 2-10x plus rapide qu'XGBoost sur les grands datasets ($> 100K$ observations). Sur notre dataset (24K) : XGBoost $F1=0.886$, LightGBM $F1=0.881$ (teste en parallele), temps XGB 2.3min vs LGB 0.8min. Avec 24K observations, la difference de vitesse est marginale. LightGBM serait prefere a partir de 500K observations.

C4.3 MLflow Prod

129 Comment enregistrez-vous un modele MLflow en production ?

Workflow MLflow : avec `mlflow.start_run()` as run: `mlflow.log_params({'n_estimators': 450, ...})`, `mlflow.log_metrics({'f1': 0.886, 'roc_auc': 0.995})`, `mlflow.xgboost.log_model(model, 'model', signature=signature)`. Le 'signature' documente le schema d'entree/sortie (types et shapes). Pour la production, on copie le modele dans le Model Registry : `mlflow.register_model('runs:/run_id/model', 'mpi_xgboost')`. On transition vers 'Production' apres validation. L'API FastAPI charge : `model = mlflow.xgboost.load_model('models:/mpi_xgboost/Production')`. Si le retraining produit un meilleur modele, on met a jour le stage 'Production' sans changer le code de l'API.

C3.2 Pearson

130 Qu'est-ce que la correlation de Pearson et ses limites pour la selection de features ?

La correlation de Pearson mesure la relation lineaire entre deux variables continues : r in $[-1, 1]$. $r=1$ = correlation parfaite positive, $r=0$ = aucune correlation lineaire. Utilise pour detecter la multicolinearite : si `feature_A` et `feature_B` ont $r > 0.95$, garder une seule. Limitation : capture uniquement les relations lineaires. Deux features non-lineairement correlee (ex: vibration et age^2 - relation quadratique) auraient $r=0$ mais sont correlees. Pour les modeles arbres, la VIF (Variance Inflation Factor) ou l'importance des features post-entrainement est plus fiable que la correlation de Pearson pour la selection.

C4.1 Robustesse Inference

131 Comment tester la robustesse du modele face a des valeurs aberrantes en inference ?

Tests de robustesse en inference : 1) Valeurs extremes : vibration = 100g (10x la valeur max de l'entrainement). Le modele XGBoost repond par la prediction de la feuille la plus proche (pas de crash), mais la prediction peut etre erronee. Guard : clip les features a $[\text{min_train} - 3 \cdot \text{std}, \text{max_train} + 3 \cdot \text{std}]$ avant inference, avec un flag `out_of_distribution=True` dans la reponse. 2) Features manquantes : si `temperature_ambient` est `None` a l'inference, on impute avec la mediane de l'entrainement (stockee dans le pipeline). 3) Input type error : pydantic valide les types (float, int) avant de passer au modele. Une string 'abc' pour vibration -> HTTP 422 avec message clair.

C4.1 Cold Start MPI

132 Comment le modele se comporte-t-il sur les machines nouvellement commissionnees ?

Le cold start problem pour les nouvelles machines : elles n'ont pas d'historique (`maintenance_age_days=0`, `rolling_failure_count_30d=0`). Impact : le modele sous-estime le risque sur les nouvelles machines (les premieres pannes surviennent souvent dans les 3 premiers mois - 'infant mortality'). Solution : 1) Periode de break-in - pendant les 90 premiers jours, la prediction est supplementee par une heuristique basee sur le fabricant (taux de panne early-life). 2) Transfer learning depuis machines similaires - si une machine du meme modele a deja un historique, utiliser ses premieres semaines comme proxy. 3) Alerte 'nouvelle machine' dans le dashboard pendant 90 jours.

C4.3 Batch vs RT

133 Qu'est-ce que le batch inference vs le real-time inference et lequel avez-vous choisi ?

Batch inference : predictions calculees periodiquement (ex: toutes les 15 minutes) sur toutes les machines. Resultats stockes en base. Latence : jusqu'a 15 minutes. Real-time inference : prediction calculee a la demande (API POST /predict) pour une machine specifique. Latence : < 100ms. Notre choix : batch inference toutes les 15 minutes (ajustable). Justification : les alertes de maintenance n'ont pas besoin d'une latence de quelques secondes (un technicien ne va pas intervenir en temps reel sur une prediction). Le batch permet de calculer SHAP pour toutes les machines a chaque run (couteux en temps reel). L'API FastAPI sert les resultats pre-calculs (lecture Redis), pas le modele directement.

C4.3 ONNX

134 Qu'est-ce que ONNX et comment accelere-t-il l'inference ?

ONNX (Open Neural Network Exchange) est un format de representation de modeles ML interoperable. XGBoost peut etre exporte en ONNX : `model.save_model('model.onnx')`. L'inference ONNX via `onnxruntime` est plus rapide que XGBoost Python car : 1) `onnxruntime` est optimise en C++ avec des instructions SIMD (vectorisation CPU). 2) Support des accelerateurs hardware : GPU (`CUDAExecutionProvider`), Intel (`OpenVINOExecutionProvider`). Benchmark sur notre modele : XGBoost Python = 8ms/prediction, ONNX Runtime = 1.2ms. Gain de 6.7x. Pour notre batch toutes les 15 minutes sur 500 machines : $500 * 8\text{ms} = 4\text{s}$ (Python) vs $500 * 1.2\text{ms} = 0.6\text{s}$ (ONNX).

C3.1 Representativite

135 Comment avez-vous valide que votre dataset est representatif ?

Validation de representativite : 1) Distribution temporelle - les 24042 observations couvrent 3 ans (uniformement). Pas de biais de sampling (pas juste les periodes de haute utilisation). 2) Distribution des types de machines - le dataset contient les memes proportions de types qu'en production (verification avec le DSI). 3) Distribution des conditions operationnelles - comparaison des distributions temperature/vibration du dataset vs des donnees de production recentes (KS test $p > 0.05$ = distribution comparable). 4) Biais de selection potentiel : les pannes sont sur-representees dans les donnees historiques car les donnees avant 2019 ne loggiaient les capteurs qu'en cas d'anomalie. Ce biais est documente dans la Model Card.

Section 10 - Perspectives, valeur industrielle et RNCP

C4.1 SHAP Fiabilité

136 Comment avez-vous mesure la fiabilité de vos interpretations SHAP ?

La fiabilité des SHAP values depend de la stabilité du modele. Validation : 1) Stabilité bootstrap - re-entraîner 10 fois avec des sous-échantillons différents. Si le ranking SHAP des features est identique dans 9/10 runs, l'interprétation est stable. Resultat : maintenance_age_days est top-1 dans 10/10 runs (très stable). vibration_mean_24h est top-3 dans 8/10 runs (stable). 2) Coherence avec la physique - les directions SHAP correspondent-elles a la physique ? (age eleve -> panne plus probable = physiquement sense). Une SHAP en contradiction avec la physique serait un signal d'artefact. 3) SHAP interaction values : vérifier que les interactions capturées (age x temperature) sont cohérentes.

C4.4 SLA

137 Qu'est-ce que la notion de 'service level agreement' (SLA) pour votre API ?

SLA pour l'API de prediction MPI : 1) Disponibilité : 99.5% (soit < 3.65h d'indisponibilité par an). 2) Latence : P95 < 150ms pour les requêtes /predict sur une machine. 3) Throughput : 50 requêtes/seconde simultanées (100 machines en batch par rafale). 4) Fraicheur : les predictions batch sont rafraichies toutes les 15 minutes (si le job echoue, alerte dans les 30 minutes). Ces SLAs sont definis avec les équipes de maintenance. Un SLA non tenu pendant 3 mois consecutifs declenche une revue architecturale. Le calcul de disponibilité exclut les fenêtrés de maintenance planifiées (dimanche 00:00-04:00).

C4.3 Retrainement

138 Comment gerez-vous le retraining periodique du modele ?

Pipeline de retraining : 1) Trigger : detection de drift ($PSI > 0.25$) ou tous les 3 mois. 2) Data refresh : charger les nouvelles observations depuis la CMMS (base de gestion de maintenance) et les capteurs des 3 derniers mois. 3) Feature engineering : meme pipeline sklearn (version versionnée dans Git). 4) Optuna : 50 nouvelles trials (le precedent point optimal est utilise comme point de depart via hyperparameter warm start). 5) Evaluation : le nouveau modele doit dépasser ancien_model_f1 - 0.01. 6) MLflow : nouveau run loggué, stage 'Staging' puis 'Production' apres validation. Tout le pipeline est orchestre par un job Python planifié (Task Scheduler, action directe python.exe).

C3.1 Stratification

139 Qu'est-ce que la stratification dans le train/test split ?

La stratification garantit que les proportions de classes sont identiques dans train, validation et test. Sans stratification sur notre dataset : un split aleatoire pourrait mettre 9.5% de pannes dans le train et 4.5% dans le test (fluctuation statistique sur 7% moyen). Cela fausserait l'évaluation. Avec stratification (stratify=y dans train_test_split) : le test contient exactement 7% de pannes, identique au train. Pour TimeSeriesSplit, la stratification n'est pas applicable (pas de melange temporel). A la place, on vérifie manuellement que chaque fold temporel contient un pourcentage de pannes comparable (entre 5% et 9%).

C3.4 Bias Variance

140 Qu'est-ce que le concept de 'bias-variance tradeoff' en pratique ?

Biais : erreur systematique due a des hypotheses trop simples (underfitting). Variance : erreur due a la sensibilité aux fluctuations du dataset d'entraînement (overfitting). Un arbre profond (max_depth=12) : faible biais, forte variance (overfitting, $F1_{test} \ll F1_{train}$). Un arbre peu profond (max_depth=3) : fort biais, faible variance (underfitting, $F1_{train}$ faible). XGBoost max_depth=6 : équilibre optimal (gap train-test = 3%). La régularisation (L1/L2, gamma) réduit la variance. Le learning_rate faible (0.08) ralentit l'apprentissage = réduit la variance. En pratique : augmenter n_estimators jusqu'au plateau puis régulariser pour réduire la variance sans dégrader le biais.

C2.3 Fixtures

141 Comment avez-vous cree votre dataset synthetique pour les tests unitaires ?

Fixture pytest avec un dataset deterministe (seed=42) : `X_test_synthetic = pd.DataFrame({'vibration_mean_24h': [0.1, 0.8, 0.5], 'temperature_delta': [2, 15, 8], 'maintenance_age_days': [10, 720, 400], ...})`. Les cas couverts : machine jeune + faible vibration -> pas de panne (expected=0), machine vieille + vibration elevee -> panne certaine (expected=1), cas ambigus (proba 0.35-0.65). Ces fixtures sont independantes du vrai dataset (pas besoin de telecharger 24042 lignes pour lancer les tests unitaires). Executee en < 1s. Les tests d'integration utilisent le vrai dataset (plus lents, skippees en mode fast).

C4.2 Feedback Loop

142 Qu'est-ce que la notion de 'feedback loop' dans les systemes ML en production ?

La feedback loop ferme le cycle : le modele predict -> les techniciens agissent ou ignorent -> la realite confirme ou infirme la prediction. Sans feedback loop, on ne peut pas mesurer la performance reelle en production. Implementation : 1) Logging des predictions (machine_id, timestamp, probabilite, label predict). 2) Apres l'evenement (panne ou non) : mise a jour de la table predictions avec outcome_actual et outcome_timestamp. 3) Calcul quotidien du F1 sur les predictions confirmees des 30 derniers jours. Probleme : le label est parfois inconnu (la machine n'est pas tombee en panne mais peut-etre aurait-elle du - intervention preventive reussie). Ce 'biais de survie inverse' est difficile a corriger.

C3.2 Feature Doc

143 Comment avez-vous documente le feature engineering pour la reproductibilite ?

Documentation en 3 couches : 1) Code autocommente - les fonctions de feature engineering ont des docstrings explicatifs (maintenance_age_days : 'Jours ecoules depuis la derniere maintenance preventive. Source: CMMS table maintenance_logs. Valeur 0 = maintenance effectuee aujourd'hui.'). 2) docs/FEATURE_DICTIONARY.md - table complete : feature_name | type | source | calcul | range_valide | importance_SHAP. 3) MLflow logging - chaque feature est loggue comme param du run (feature_count=15, features=['maintenance_age_days', ...]). La reproductibilite complete : quelqu'un avec acces a la CMMS peut recreer le feature engineering exactement depuis FEATURE_DICTIONARY.md.

C3.1 Ground Truth

144 Qu'est-ce que la notion de 'ground truth' et comment l'avez-vous etablie ?

La ground truth designe les vraies etiquettes (la realite). Pour la maintenance predictive : label=1 si la machine est tombee en panne dans les 24h suivant l'observation. Source de verite : la CMMS (Computerized Maintenance Management System) qui enregistre chaque incident technique avec timestamp et machine_id. La definition exacte de 'panne' est cruciale : on a exclu les arrêts planifies (maintenance preventive) et les arrêts administratifs (manque de commandes). Seuls les arrêts non planifies avec intervention technique sont labelises 1. Cette definition precise a ete validee avec le responsable maintenance avant le debut du projet.

C4.4 SRP ML

145 Comment avez-vous applique le principe de responsabilite unique au code ML ?

Separation des concerns dans le projet MPI : data_loader.py (chargement + validation donnees, rien d'autre), feature_engineering.py (transformations, pas d'accès BD), trainer.py (entrainement + evaluation, pas de feature engineering), predictor.py (inference uniquement, charge le modele serialise), monitoring.py (metriques de drift, separation totale du pipeline d'entrainement). Chaque module est testable independamment. Exemple : feature_engineering.py prend un DataFrame et retourne un DataFrame (pur, pas de side effects). On peut tester le feature engineering avec des donnees synthetiques sans toucher a la BD.

C4.4 Dead Code

146 Qu'est-ce que le concept de 'dead code' et comment l'elimineriez-vous dans ce projet ?

Dead code : code presente dans le repo mais jamais execute (fonctions inutilisees, branches conditionnelles impossibles, imports non utilises). Outils de detection : vulture (Python) = detecte les fonctions/variables non references, flake8 F401 = imports non utilises, coverage.py = lignes jamais executees par les tests. Dans MPI, on a identifie 2 modeles experimentaux (SVM, KNN) abandonnes apres le benchmark mais encore presents dans models/experimental.py. Decision : les supprimer ou les deplacer dans une branche archive. Rule of thumb : si un code n'est pas teste, il est potentiellement dead. La dette technique du dead code = maintenance inutile et confusion pour les nouveaux contributeurs.

C4.3 ONNX Validation

147 Comment avez-vous valide le deployment ONNX avec des tests de bout en bout ?

Pipeline de validation ONNX : 1) Export : `model.save_model('model.onnx')`. 2) Consistency check : pour 1000 observations du test set, comparer les probabilites XGBoost Python vs ONNX Runtime. La difference maximale acceptable est $1e-5$ (arrondi flottant). 3) Latency benchmark : 100 predictions sequentielles et mesure P50/P99. 4) Integration test : l'API FastAPI charge le modele ONNX et repond correctement a une requete POST /predict avec un vrai payload. 5) Shadow test : 1 semaine en parallel avec le modele Python, comparaison des alertes generees. Aucune difference notable -> promotion ONNX en production.

C4.3 Model Versioning

148 Comment gerez-vous la version du modele dans le code applicatif ?

Versioning explicite : le nom du fichier modele inclut la version (`mpi_xgboost_v1.2.3.onnx`) et le hash SHA-256 du fichier est store en config. L'API FastAPI au demarrage : 1) Charge le modele depuis le chemin configures en env var `MODEL_PATH`. 2) Verifie le SHA-256 attendu (`MODEL_HASH` env var). 3) Log la version et la date d'entrainement depuis le model metadata. Ainsi, si le mauvais modele est deploye par erreur, le hash check fail au demarrage avec une erreur claire. Dans MLflow : `model_version=1.2.3` est tag sur le run. Le Streamlit dashboard affiche 'Modele v1.2.3, entraine le 2025-11-15' pour la tracabilite.

C3.5 Adversarial

149 Qu'est-ce que l'adversarial testing pour un modele de maintenance predictive ?

L'adversarial testing genere intentionnellement des inputs difficiles ou pathologiques pour tester la robustesse du modele. Exemples : 1) Monotonicity test : augmenter `maintenance_age_days` de 0 a 1000 par pas de 10 -> la probabilite de panne doit etre monotone croissante (pas decroissante). Si ce n'est pas le cas, le modele a appris une anomalie. 2) Boundary test : exactement au seuil de decision (`proba = 0.350` -> classe 1, `proba = 0.349` -> classe 0). 3) Contradiction test : `maintenance_age_days=0` + `vibration=9.9g` -> le modele doit etre incertain (pas predit avec confiance). Ces tests revelent les biais du modele que le test set standard ne detecte pas.

C4.4 Pitch DG

150 Comment presenteriez-vous ce projet en 2 minutes a un directeur general ?

Pitch 2 minutes : 'Notre systeme predit les pannes de machines industrielles 18 heures a l'avance avec 91% de fiabilite. Pour chaque EUR investi dans le systeme, il en rapporte 113 en evitant des arrets non planifies. Concretement : sur un parc de 50 machines, le systeme envoie une alerte chaque matin aux techniciens avec la liste des 3-5 machines a inspecter en priorite et une explication visuelle du pourquoi (age, vibrations anormales). En 5 mois, notre equipe de 2 etudiants a construit et valide ce systeme sur 24 000 donnees historiques reelles. La prochaine etape est le pilote sur 10 machines reelles pendant 3 mois.'